The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

The Concise TypeScript Book ofrece una visión general completa y concisa de las capacidades de TypeScript. Incluye explicaciones claras que abarcan todos los aspectos de la versión más reciente del lenguaje, desde su potente sistema de tipos hasta las características avanzadas.

Tanto si eres principiante como si tienes experiencia en el desarrollo, este libro es un recurso de gran valor para mejorar tu comprensión y dominio de TypeScript.

Este libro es completamente gratuito y de código abierto.

Creo que la formación técnica de alta calidad debe estar al alcance de todo el mundo. Por esta razón, mantengo el libro disponible gratuitamente y lo actualizo periódicamente con mejoras y nuevos ejemplos.

Descubre **The Concise TypeScript Book Plus Edition**.



Para quienes deseen ir más allá de la edición de código abierto, **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** incluye contenido adicional y exclusivo centrado en la aplicación práctica.

La Edición Plus incluye:

- **Actualizada para TypeScript 7** — cobertura de las últimas funciones y mejoras del lenguaje de TypeScript 7.
- **TypeScript con React** — orientación práctica para tipar componentes, props, hooks, eventos, children, refs y patrones habituales de React.

- **Recetas de TypeScript para proyectos reales** — ejemplos específicos que abordan problemas prácticos que los desarrolladores encuentran al crear y mantener aplicaciones TypeScript.

Al comprar la Edición Plus, también apoyas directamente el desarrollo y mantenimiento continuos del libro gratuito y de código abierto.

La Edición Plus está disponible en inglés e italiano en Amazon en todo el mundo. Descubre la Edición Plus y cómprala en Amazon.

Apoya el proyecto

Si el libro gratuito te ha ayudado a corregir un error, comprender un concepto difícil o avanzar en tu carrera profesional, considera apoyar mi trabajo pagando lo que quieras, con una contribución sugerida de **$5**, o invitándome a un café.

Tu apoyo me ayuda a mantener el contenido actualizado y ampliarlo con nuevos ejemplos, explicaciones más claras y orientación práctica adicional.





Traducciones

Este libro se ha traducido a varios idiomas, entre ellos:

Inglés

Búlgaro

Alemán

Francés

Indonesio

Italiano

Japonés

Coreano

Polaco

Portugués (Brasil)

Sueco

Turco

Vietnamese

Chino

Español

Tailandés

Ruso

Árabe Checo

Descargas y sitio web

También puedes descargar la versión EPUB:

https://github.com/gibbok/typescript-book/tree/main/downloads

Hay disponible una versión en línea en:

https://gibbok.github.io/typescript-book

Índice

- The Concise TypeScript Book
  - Apoya el proyecto
  - Traducciones
  - Descargas y sitio web
  - Índice
  - Introducción
  - Sobre el autor
  - Introducción a TypeScript
    - ¿Qué es TypeScript?
    - ¿Por qué TypeScript?
    - TypeScript y JavaScript
    - Generación de código de TypeScript
    - JavaScript moderno ahora (reducción de nivel)
  - Primeros pasos con TypeScript
    - Instalación
    - Configuración
    - Archivo de configuración de TypeScript
      - target
      - lib
      - strict

        - module
        - moduleResolution
        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Consejos para migrar a TypeScript
- Exploración del sistema de tipos
    - El servicio del lenguaje de TypeScript
    - Tipado estructural
    - Reglas fundamentales de comparación de TypeScript
    - Los tipos como conjuntos
    - Asignar un tipo: declaraciones y aserciones de tipo
        - Declaración de tipo
        - Aserción de tipo
        - Declaraciones de ambiente
    - Comprobación de propiedades y de propiedades adicionales
    - Tipos débiles
    - Comprobación estricta de literales de objeto (frescura)
    - Inferencia de tipos
    - Inferencias más avanzadas
    - Ampliación de tipos
    - Const
        - Modificador const en parámetros de tipo
        - Aserción const
    - Anotación de tipo explícita

        - Restricción de tipos
            - Condiciones
            - Lanzar una excepción o devolver
            - Unión discriminada
            - Guardas de tipo definidas por el usuario
            - Restricción mediante switch-true
    - Tipos primitivos
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null y undefined
        - Array
        - any
    - Anotaciones de tipo
    - Propiedades opcionales
    - Propiedades de solo lectura
    - Firmas de índice
    - Extensión de tipos
    - Tipos literales
    - Inferencia de literales
    - strictNullChecks
    - Enums
        - Enums numéricos
        - Enums de cadena
        - Enums constantes
        - Mapeo inverso
        - Enums de ambiente
        - Miembros calculados y constantes
    - Restricción
        - Guardas de tipo typeof

        - Restricción por veracidad
        - Restricción por igualdad
        - Restricción mediante el operador in
        - Restricción mediante instanceof
    - Asignaciones
    - Análisis del flujo de control
    - Predicados de tipo
    - Uniones discriminadas
    - El tipo never
    - Comprobación exhaustiva
    - Tipos de objeto
    - Tipo de tupla (anónima)
    - Tipo de tupla con nombres (etiquetada)
    - Tupla de longitud fija
    - Tipo de unión
    - Tipos de intersección
    - Indexación de tipos
    - Tipo a partir de un valor
    - Tipo a partir del retorno de una función
    - Tipo a partir de un módulo
    - Tipos mapeados
    - Modificadores de tipos mapeados
    - Tipos condicionales
    - Tipos condicionales distributivos
    - Inferencia de tipos con infer en tipos condicionales
    - Tipos condicionales predefinidos
    - Tipos de unión de plantilla
    - Tipo any
    - Tipo unknown
    - Tipo void
    - Tipo never
    - Interfaz y tipo

        - Sintaxis habitual
        - Tipos básicos
        - Objetos e interfaces
        - Tipos de unión e intersección
    - Tipos primitivos integrados
    - Objetos JS integrados habituales
    - Sobrecargas
    - Combinación y extensión
    - Diferencias entre tipo e interfaz
    - Clase
        - Sintaxis habitual de las clases
        - Constructor
        - Constructores privados y protegidos
        - Modificadores de acceso
        - Get y set
        - Autoaccesores en clases
        - this
        - Propiedades de parámetros
        - Clases abstractas
        - Con genéricos
        - Decoradores
            - Decoradores de clase
            - Decorador de propiedades
            - Decorador de métodos
            - Decoradores de getters y setters
            - Metadatos de decoradores
        - Herencia
        - Miembros estáticos
        - Inicialización de propiedades
        - Sobrecarga de métodos
    - Genéricos
        - Tipo genérico

        - Clases genéricas
        - Restricciones genéricas
        - Restricción contextual de genéricos
    - Tipos estructurales eliminados
    - Espacios de nombres
    - Símbolos
    - Directivas de triple barra
    - Manipulación de tipos
        - Creación de tipos a partir de tipos
        - Tipos de acceso indexado
        - Tipos de utilidad
            - Awaited<T>
            - Partial<T>
            - Required<T>
            - Readonly<T>
            - Record<K, T>
            - Pick<T, K>
            - Omit<T, K>
            - Exclude<T, U>
            - Extract<T, U>
            - NonNullable<T>
            - Parameters<T>
            - ConstructorParameters<T>
            - ReturnType<T>
            - InstanceType<T>
            - ThisParameterType<T>
            - OmitThisParameter<T>
            - ThisType<T>
            - Uppercase<T>
            - Lowercase<T>
            - Capitalize<T>
            - Uncapitalize<T>

        - NoInfer<T>
- Otros
    - Gestión de errores y excepciones
    - Clases mixin
    - Características asíncronas del lenguaje
    - Iteradores y generadores
    - Referencia de TsDocs y JSDoc
    - @types
    - JSX
    - Módulos ES6
    - Operador de exponenciación de ES7
    - La sentencia for-await-of
    - Metapropiedad new target
    - Expresiones de importación dinámica
    - “tsc –watch”
    - Operador de aserción no nula
    - Declaraciones con valores predeterminados
    - Encadenamiento opcional
    - Operador de fusión de valores nulos
    - Tipos literales de plantilla
    - Sobrecarga de funciones
    - Tipos recursivos
    - Tipos condicionales recursivos
    - Compatibilidad con módulos ECMAScript en Node
    - Funciones de aserción
    - Tipos de tupla variádicos
    - Tipos envoltorio
    - Covarianza y contravarianza en TypeScript
        - Anotaciones opcionales de varianza para parámetros de tipo
    - Firmas de índice con patrones de cadenas de plantilla

- El operador satisfies
- Importaciones y exportaciones solo de tipos
- Declaración using y gestión explícita de recursos
  - Declaración await using
- Atributos de importación
- Comprobación de la sintaxis de expresiones regulares
- import defer

Introducción

¡Te damos la bienvenida a The Concise TypeScript Book! Esta guía te proporciona los conocimientos esenciales y las habilidades prácticas necesarias para trabajar eficazmente con TypeScript. Descubre conceptos y técnicas fundamentales para escribir código limpio y robusto. Tanto si eres principiante como si tienes experiencia en el desarrollo, este libro sirve a la vez como guía completa y como referencia práctica para aprovechar la potencia de TypeScript en tus proyectos.

Este libro trata TypeScript 7.0.

Sobre el autor

Simone Poggiali es un Staff Engineer con experiencia y pasión por escribir código de calidad profesional desde la década de 1990. A lo largo de su carrera internacional, ha contribuido a numerosos proyectos para una amplia variedad de clientes, desde startups hasta grandes organizaciones. Empresas destacadas como HelloFresh, Siemens, O2, Leroy Merlin y Snowplow se han beneficiado de su experiencia y dedicación.

Puedes contactar con Simone Poggiali en las siguientes plataformas:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- Correo electrónico: gibbok.coding gmail.com

Lista completa de colaboradores: https://github.com/gibbok/typescript-book/graphs/contributors

Introducción a TypeScript

¿Qué es TypeScript?

TypeScript es un lenguaje de programación con tipado fuerte que se basa en JavaScript. Anders Hejlsberg lo diseñó originalmente en 2012 y Microsoft lo desarrolla y mantiene actualmente como proyecto de código abierto.

TypeScript se compila en JavaScript y puede ejecutarse en cualquier entorno de ejecución de JavaScript (por ejemplo, un navegador o Node.js en un servidor).

Admite varios paradigmas de programación, como la programación funcional, genérica, imperativa y orientada a objetos, y es un lenguaje compilado (transpilado) que se convierte a JavaScript antes de su ejecución.

¿Por qué TypeScript?

TypeScript es un lenguaje con tipado fuerte que ayuda a prevenir errores habituales de programación y a evitar

determinados tipos de errores en tiempo de ejecución antes de ejecutar el programa.

Un lenguaje con tipado fuerte permite al desarrollador especificar distintas restricciones y comportamientos del programa en las definiciones de tipos de datos, lo que facilita verificar que el software sea correcto y prevenir defectos. Esto resulta especialmente útil en aplicaciones a gran escala.

Algunas de las ventajas de TypeScript:

- Tipado estático, opcionalmente con tipado fuerte
- Inferencia de tipos
- Acceso a las características de ES6 y ES7
- Compatibilidad multiplataforma y entre navegadores
- Compatibilidad de herramientas con IntelliSense

TypeScript y JavaScript

TypeScript se escribe en archivos `.ts` o `.tsx`, mientras que JavaScript se escribe en archivos `.js` o `.jsx`.

Los archivos con la extensión `.tsx` o `.jsx` pueden contener la extensión de sintaxis JSX de JavaScript, que se utiliza en React para desarrollar interfaces de usuario.

En cuanto a la sintaxis, TypeScript es un superconjunto tipado de JavaScript (ECMAScript 2015). Todo código JavaScript es código TypeScript válido, pero lo contrario no siempre es cierto.

Por ejemplo, considera una función en un archivo JavaScript con la extensión `.js`, como la siguiente:

```
const sum = (a, b) => a + b;
```

La función puede convertirse y utilizarse en TypeScript cambiando la extensión del archivo a `.ts`. Sin embargo, si la misma función se anota con tipos de TypeScript, no puede ejecutarse en ningún entorno de ejecución de JavaScript sin compilarla. El siguiente código TypeScript producirá un error de sintaxis si no se compila:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript se diseñó para detectar posibles errores en tiempo de ejecución durante la compilación, ya que permite a los desarrolladores expresar su intención mediante anotaciones de tipo. Además, gracias a la inferencia de tipos, TypeScript también puede detectar determinados problemas aunque no se proporcionen anotaciones de tipo explícitas. Por ejemplo, el siguiente fragmento de código no especifica ningún tipo de TypeScript:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

En este caso, TypeScript detecta un error e informa de lo siguiente:

```
Property 'y' does not exist on type '{ x: number; }'.
```

El sistema de tipos de TypeScript está influido en gran medida por el comportamiento de JavaScript en tiempo de ejecución. Por ejemplo, el operador de suma (+), que en JavaScript puede realizar tanto una concatenación de cadenas como una suma numérica, se modela del mismo modo en TypeScript:

```
const result = '1' + 1; // Result is of type string
```

El equipo responsable de TypeScript tomó la decisión deliberada de señalar como errores los usos inusuales de JavaScript. Por ejemplo, considera el siguiente código JavaScript válido:

```
const result = 1 + true; // In JavaScript, the result is equal to 2
```

Sin embargo, TypeScript genera un error:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Este error se produce porque TypeScript aplica estrictamente la compatibilidad de tipos y, en este caso, identifica una operación no válida entre un número y un booleano.

Generación de código de TypeScript

El compilador de TypeScript tiene dos responsabilidades principales: comprobar si hay errores de tipos y compilar en JavaScript. Estos dos procesos son independientes entre sí. Los tipos no afectan a la ejecución del código en un entorno de ejecución de JavaScript, ya que se eliminan por completo durante la compilación. TypeScript puede generar JavaScript incluso cuando existen errores de tipos. Este es un ejemplo de código TypeScript con un error de tipos:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Argument of type 'string' is not assignable to parameter of type 'number'.
```

Sin embargo, aún puede producir una salida de JavaScript ejecutable:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

No es posible comprobar los tipos de TypeScript en tiempo de ejecución. Por ejemplo:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' only refers to a type, but is being used as a
value here.
        // ...
    }
};
```

Como los tipos se eliminan después de la compilación, no hay forma de ejecutar este código en JavaScript. Para reconocer tipos en tiempo de ejecución, debemos utilizar otro mecanismo. TypeScript ofrece varias opciones; una de las más habituales es la «unión etiquetada». Por ejemplo:

```
interface Dog {
    kind: 'dog'; // Tagged union
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Tagged union
    meow: () => void;
}
```

```ts
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

La propiedad “kind” es un valor que puede utilizarse en tiempo de ejecución para distinguir objetos en JavaScript.

También es posible que un valor tenga en tiempo de ejecución un tipo distinto del declarado en la declaración de tipo. Por ejemplo, si el desarrollador ha interpretado incorrectamente el tipo de una API y lo ha anotado de forma errónea.

TypeScript es un superconjunto de JavaScript, por lo que la palabra clave “class” puede utilizarse como tipo y como valor en tiempo de ejecución.

```ts
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
```

```
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

En JavaScript, una “class” tiene una propiedad “prototype”, y el operador “instanceof” puede utilizarse para comprobar si la propiedad prototype de un constructor aparece en algún punto de la cadena de prototipos de un objeto.

TypeScript no afecta al rendimiento en tiempo de ejecución, ya que todos los tipos se eliminan. Sin embargo, TypeScript sí añade cierta sobrecarga al tiempo de compilación.

JavaScript moderno ahora (reducción de nivel)

TypeScript puede compilar código para cualquier versión publicada de JavaScript desde ECMAScript 3 (1999). Esto

significa que TypeScript puede transpilar código que utiliza las características más recientes de JavaScript a versiones anteriores, un proceso conocido como reducción de nivel. Esto permite utilizar JavaScript moderno y mantener a la vez la máxima compatibilidad con entornos de ejecución antiguos.

Es importante tener en cuenta que, al transpilar a una versión anterior de JavaScript, TypeScript puede generar código que suponga una sobrecarga de rendimiento en comparación con las implementaciones nativas.

Estas son algunas de las características modernas de JavaScript que pueden utilizarse en TypeScript:

- Módulos de ECMAScript en lugar de callbacks “define” al estilo de AMD o sentencias “require” de CommonJS.
- Clases en lugar de prototipos.
- Declaración de variables mediante “let” o “const” en lugar de “var”.
- Bucle “for-of” o “.forEach” en lugar del bucle “for” tradicional.
- Funciones flecha en lugar de expresiones de función.
- Asignación mediante desestructuración.
- Nombres abreviados de propiedades o métodos y nombres de propiedades calculados.
- Parámetros de función predeterminados.

Al aprovechar estas características modernas de JavaScript, los desarrolladores pueden escribir código TypeScript más expresivo y conciso.

Primeros pasos con TypeScript

Instalación

Visual Studio Code ofrece una excelente compatibilidad con el lenguaje TypeScript, pero no incluye el compilador de TypeScript. Para instalarlo, puedes utilizar un gestor de paquetes como npm o yarn:

```
npm install typescript --save-dev
```

o

```
yarn add typescript --dev
```

Asegúrate de incluir en el repositorio el archivo de bloqueo generado para garantizar que todos los miembros del equipo utilicen la misma versión de TypeScript.

Para ejecutar el compilador de TypeScript, puedes utilizar los siguientes comandos:

```
npx tsc
```

o

```
yarn tsc
```

Se recomienda instalar TypeScript en cada proyecto en lugar de hacerlo globalmente, ya que así el proceso de compilación es más predecible. No obstante, para usos puntuales puedes utilizar el siguiente comando:

```
npx tsc
```

o instalarlo globalmente:

```
npm install -g typescript
```

Si utilizas Microsoft Visual Studio, puedes obtener TypeScript como paquete de NuGet para tus proyectos de MSBuild. Ejecuta el siguiente comando en la consola del Administrador de paquetes NuGet:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Durante la instalación de TypeScript se instalan dos ejecutables: “tsc”, el compilador de TypeScript, y “tsserver”, el servidor independiente de TypeScript. El servidor independiente contiene el compilador y los servicios del lenguaje que los editores y los IDE pueden utilizar para ofrecer finalización inteligente de código.

Además, existen varios transpiladores compatibles con TypeScript, como Babel (mediante un complemento) o swc. Estos transpiladores pueden utilizarse para convertir código TypeScript a otros lenguajes o versiones de destino.

TypeScript 7.0 se reescribió en Go como implementación nativa del compilador y del servicio del lenguaje. Utiliza multihilo con memoria compartida y otras optimizaciones para acelerar las compilaciones completas y las funciones del editor, lo que reduce el tiempo de respuesta durante el desarrollo.

Algunas características de rendimiento de TypeScript 7.0 pueden ajustarse. La comprobación de tipos puede ejecutarse en procesos paralelos mediante `--checkers`; un mayor número de procesos puede acelerar proyectos grandes, pero consume más memoria. El modo `--watch` reconstruido mejora la supervisión de archivos entre plataformas. TypeScript 7.0 aún no incluye una API del compilador (a fecha de julio de 2026), por lo que las herramientas que todavía necesiten la API de

TypeScript 6.0 pueden ejecutarse junto con TypeScript 7.0 mediante `@typescript/typescript6` o alias de npm.

Configuración

TypeScript puede configurarse mediante las opciones de la CLI de tsc o con un archivo de configuración específico llamado tsconfig.json situado en la raíz del proyecto.

Para generar un archivo tsconfig.json con los ajustes recomendados ya incluidos, puedes utilizar el siguiente comando:

```
tsc --init
```

Al ejecutar localmente el comando `tsc`, TypeScript compilará el código con la configuración especificada en el archivo tsconfig.json más cercano.

Estos son algunos ejemplos de comandos de la CLI que se ejecutan con la configuración predeterminada:

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

Archivo de configuración de TypeScript

Un archivo tsconfig.json sirve para configurar el compilador de TypeScript (tsc). Normalmente se añade a la raíz del proyecto junto con el archivo `package.json`.

Notas:

- tsconfig.json admite comentarios aunque tenga formato JSON.
- Es aconsejable utilizar este archivo de configuración en lugar de las opciones de la línea de comandos.

En los siguientes enlaces encontrarás la documentación completa y su esquema:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

La siguiente lista recoge algunas configuraciones habituales y útiles:

target

La propiedad “target” especifica la versión de ECMAScript a la que debe emitirse o compilarse el código TypeScript. Para los navegadores modernos, ES6 es una buena opción. Nota: la compatibilidad con ES5 quedó obsoleta en TypeScript 6.0 y ya no se admite en TypeScript 7.0.

lib

La propiedad “lib” especifica qué archivos de biblioteca deben incluirse durante la compilación. TypeScript incluye automáticamente las API de las características indicadas en la propiedad “target”, pero es posible omitir o seleccionar bibliotecas concretas para necesidades específicas. Por ejemplo, si trabajas en un proyecto de servidor, podrías excluir la

biblioteca “DOM”, que solo resulta útil en un entorno de navegador.

strict

La opción “strict” mejora la seguridad de tipos al activar comprobaciones más rigurosas. Está activada de forma predeterminada desde TypeScript 6.0; en versiones anteriores, debes establecerla explícitamente en true en tsconfig.json. Al activar “strict”, TypeScript puede:

- Emitir código mediante “use strict” para cada archivo fuente.
- Tener en cuenta “null” y “undefined” durante la comprobación de tipos.
- Deshabilitar el uso del tipo “any” cuando no haya anotaciones de tipo.
- Generar un error al utilizar la expresión “this”, que de otro modo implicaría el tipo “any”.

module

La propiedad “module” establece el sistema de módulos compatible con el programa compilado. En tiempo de ejecución se utiliza un cargador de módulos para localizar y ejecutar las dependencias según el sistema especificado.

Los cargadores de módulos más habituales en JavaScript son CommonJS de Node.js para aplicaciones del lado del servidor y RequireJS para módulos AMD en aplicaciones web ejecutadas en el navegador. TypeScript puede emitir código para varios sistemas de módulos, entre ellos UMD, System, ESNext,

ES2015/ES6 y ES2020. El sistema de módulos debe elegirse según el entorno de destino y el mecanismo de carga disponible en él.

Nota: la compatibilidad con sistemas de módulos antiguos (AMD, UMD y SystemJS) quedó obsoleta en TypeScript 6.0 y ya no se admite en TypeScript 7.0.

moduleResolution

La propiedad “moduleResolution” especifica la estrategia de resolución de módulos. Utiliza “nodenext” o “bundler” para código TypeScript moderno. La estrategia “classic” solo se utiliza con versiones antiguas de TypeScript (anteriores a la 1.6).

esModuleInterop

La propiedad “esModuleInterop” permite importaciones predeterminadas desde módulos CommonJS que no exportan mediante la propiedad “default”; esta propiedad proporciona un adaptador para garantizar la compatibilidad del JavaScript emitido. Tras activar esta opción, podemos utilizar `import MyLibrary from "my-library"` en lugar de `import * as MyLibrary from "my-library"`.

“esModuleInterop” era originalmente opcional para evitar cambios incompatibles, pero desde hace tiempo es el valor predeterminado recomendado. Desactivarlo puede provocar problemas sutiles en tiempo de ejecución al utilizar CommonJS con ESM. Nota: desde TypeScript 6.0, este comportamiento de interoperabilidad más seguro está siempre activado.

En TypeScript 6.0, algunas opciones de configuración y formas sintácticas antiguas quedaron obsoletas o pasaron por un periodo de compatibilidad con el comportamiento anterior. En TypeScript 7.0, producen errores no recuperables o no tienen efecto.

Las características obsoletas que se han convertido en errores no recuperables o en comportamientos sin efecto son:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- desactivar `esModuleInterop` o `allowSyntheticDefaultImports`
- desactivar `alwaysStrict`
- la palabra clave `module` en declaraciones de espacios de nombres
- `asserts` en importaciones
- `/// <reference no-default-lib />` con `skipDefaultLibCheck`
- rutas de archivos de la CLI con un `tsconfig.json` local, salvo que se utilice `--ignoreConfig`

jsx

La propiedad “jsx” solo se aplica a los archivos .tsx utilizados en ReactJS y controla cómo se compilan a JavaScript las construcciones JSX. Una opción habitual es “preserve”, que compila a un archivo .jsx manteniendo JSX sin cambios para que pueda pasarse a otras herramientas, como Babel, y aplicar transformaciones adicionales.

skipLibCheck

La propiedad “skipLibCheck” impide que TypeScript compruebe los tipos de todos los paquetes de terceros importados. Esta propiedad reduce el tiempo de compilación de un proyecto. TypeScript seguirá comprobando tu código con las definiciones de tipos proporcionadas por esos paquetes.

files

La propiedad “files” indica al compilador una lista de archivos que siempre deben incluirse en el programa.

include

La propiedad “include” indica al compilador una lista de archivos que queremos incluir. Esta propiedad admite patrones similares a glob, como “*” *para cualquier subdirectorio,* ”” para cualquier nombre de archivo y “?” para caracteres opcionales.

exclude

La propiedad “exclude” indica al compilador una lista de archivos que no deben incluirse en la compilación. Puede incluir archivos como “node_modules” o archivos de pruebas. Nota: tsconfig.json admite comentarios.

importHelpers

TypeScript utiliza código auxiliar al generar código para determinadas características avanzadas o reducidas de nivel de JavaScript. De forma predeterminada, estos auxiliares se

duplican en los archivos que los utilizan. La opción `importHelpers` los importa en su lugar desde el módulo `tslib`, lo que hace más eficiente la salida de JavaScript.

Consejos para migrar a TypeScript

En proyectos grandes se recomienda adoptar una transición gradual en la que inicialmente coexistan el código TypeScript y el código JavaScript. Solo los proyectos pequeños pueden migrarse a TypeScript de una sola vez.

El primer paso de esta transición consiste en introducir TypeScript en el proceso de la cadena de compilación. Puede hacerse mediante la opción del compilador "allowJs", que permite que los archivos .ts y .tsx coexistan con los archivos JavaScript existentes. Como TypeScript recurrirá al tipo "any" para una variable cuando no pueda inferir su tipo a partir de archivos JavaScript, se recomienda desactivar "noImplicitAny" en las opciones del compilador al principio de la migración.

El segundo paso consiste en asegurarte de que las pruebas de JavaScript funcionen junto con los archivos TypeScript, de modo que puedas ejecutar las pruebas a medida que conviertes cada módulo. Si utilizas Jest, considera usar `ts-jest`, que permite probar proyectos TypeScript con Jest.

El tercer paso consiste en incluir en el proyecto las declaraciones de tipos de las bibliotecas de terceros. Estas declaraciones pueden encontrarse incluidas con las bibliotecas o en DefinitelyTyped. Puedes buscarlas en https://www.typescriptlang.org/dt/search e instalarlas mediante:

```
npm install --save-dev @types/package-name
```

o

```
yarn add --dev @types/package-name
```

El cuarto paso consiste en migrar módulo por módulo con un enfoque ascendente, siguiendo el grafo de dependencias y comenzando por las hojas. La idea es empezar convirtiendo los módulos que no dependen de otros. Para visualizar los grafos de dependencias, puedes utilizar la herramienta “madge”.

Las funciones auxiliares y el código relacionado con API o especificaciones externas son buenos candidatos para estas conversiones iniciales. Es posible generar automáticamente definiciones de tipos de TypeScript a partir de contratos de Swagger o esquemas de GraphQL o JSON e incluirlas en el proyecto.

Cuando no haya especificaciones ni esquemas oficiales disponibles, puedes generar tipos a partir de datos sin procesar, como el JSON devuelto por un servidor. Sin embargo, se recomienda generar los tipos a partir de especificaciones y no de datos para evitar omitir casos límite.

Durante la migración, evita refactorizar el código y céntrate exclusivamente en añadir tipos a los módulos.

El quinto paso consiste en activar “noImplicitAny”, que exigirá que todos los tipos se conozcan y estén definidos, y ofrecerá una mejor experiencia con TypeScript en el proyecto.

Durante la migración puedes utilizar la directiva `@ts-check`, que activa la comprobación de tipos de TypeScript en un archivo JavaScript. Esta directiva proporciona una versión flexible de la comprobación de tipos y puede utilizarse inicialmente para

identificar problemas en los archivos JavaScript. Cuando un archivo incluye `@ts-check`, TypeScript intenta deducir definiciones mediante comentarios de estilo JSDoc. No obstante, considera utilizar anotaciones JSDoc únicamente en una fase muy temprana de la migración.

Considera mantener en false el valor predeterminado de `noEmitOnError` en tsconfig.json. Esto te permitirá emitir código fuente JavaScript aunque se notifiquen errores.

Exploración del sistema de tipos

El servicio del lenguaje de TypeScript

El servicio del lenguaje de TypeScript, también conocido como tsserver, ofrece distintas funciones, como informes de errores, diagnósticos, compilación al guardar, cambio de nombre, navegación a la definición, listas de finalización, ayuda de firmas y muchas más. Lo utilizan principalmente los entornos de desarrollo integrados (IDE) para ofrecer compatibilidad con IntelliSense. Se integra a la perfección con Visual Studio Code y lo emplean herramientas como Conquer of Completion (Coc).

Los desarrolladores pueden aprovechar una API específica y crear sus propios complementos personalizados para el servicio del lenguaje con el fin de mejorar la experiencia de edición de TypeScript. Esto puede ser especialmente útil para implementar funciones específicas de linting o activar la finalización automática para un lenguaje de plantillas personalizado.

Un ejemplo real de complemento personalizado es “typescript-styled-plugin”, que ofrece informes de errores de sintaxis y

compatibilidad con IntelliSense para propiedades CSS en componentes con estilo.

Para obtener más información y consultar guías de inicio rápido, puedes acudir a la wiki oficial de TypeScript en GitHub: https://github.com/microsoft/TypeScript/wiki/

Tipado estructural

TypeScript se basa en un sistema de tipos estructural. Esto significa que la compatibilidad y la equivalencia de los tipos se determinan por su estructura o definición real, y no por su nombre o lugar de declaración, como ocurre en sistemas de tipos nominales como C# o C.

El sistema de tipos estructural de TypeScript se diseñó a partir del funcionamiento en tiempo de ejecución del sistema dinámico de tipado pato de JavaScript.

El siguiente ejemplo es código TypeScript válido. Como puedes observar, “X” e “Y” tienen el mismo miembro “a”, aunque sus declaraciones tengan nombres distintos. Los tipos vienen determinados por sus estructuras y, en este caso, como las estructuras son iguales, son compatibles y válidos.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Valid
```

Reglas fundamentales de comparación de TypeScript

El proceso de comparación de TypeScript es recursivo y se ejecuta sobre tipos anidados a cualquier nivel.

Un tipo “X” es compatible con “Y” si “Y” tiene, como mínimo, los mismos miembros que “X”.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Valid, as it has at least the
same members as X
const r: X = y;
```

Los parámetros de las funciones se comparan por sus tipos, no por sus nombres:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Valid
x = y; // Valid
```

Los tipos de retorno de las funciones deben ser iguales:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Invalid
x = y; // Invalid
```

El tipo de retorno de una función de origen debe ser un subtipo del tipo de retorno de una función de destino:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
```

```
x = y; // Valid
y = x; // Invalid member b is missing
```

Se permite descartar parámetros de funciones, ya que es una práctica habitual en JavaScript, por ejemplo al utilizar “Array.prototype.map()”:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Por tanto, las siguientes declaraciones de tipos son totalmente válidas:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Missing b parameter
y = x; // Valid
```

Cualquier parámetro opcional adicional del tipo de origen es válido:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; //Valid
```

Los parámetros opcionales del tipo de destino que no tengan parámetros correspondientes en el tipo de origen son válidos y no constituyen un error:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
```

```
y = x; // Valid
x = y; // Valid
```

El parámetro rest se trata como una serie infinita de parámetros opcionales:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //valid
```

Las funciones con sobrecargas son válidas si la firma de sobrecarga es compatible con su firma de implementación:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Valid
x('a', 1); // Valid

function y(a: string): void; // Invalid, not compatible with implementation signature
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

La comparación de parámetros de funciones tiene éxito si los parámetros de origen y destino pueden asignarse a supertipos o subtipos (bivarianza).

```
// Supertype
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
```

```
    }
}
// Subtype
class Y extends X {}
// Subtype
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Bivariance does accept supertypes
console.log(getA(new X('x'))); // Valid
console.log(getA(new Y('Y'))); // Valid
console.log(getA(new Z('z'))); // Valid
```

Los enums pueden compararse con números y asignarse a ellos, y viceversa, pero no es válido comparar valores de enums de tipos distintos.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Valid
const ya: Y = 0; // Valid
X.A === Y.A; // Invalid
```

Las instancias de una clase se someten a una comprobación de compatibilidad de sus miembros privados y protegidos:

```
class X {
    public a: string;
    constructor(value: string) {
```

```
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Invalid
```

La comprobación de compatibilidad no tiene en cuenta las distintas jerarquías de herencia. Por ejemplo:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
```

```
x === y; // Valid
x === z; // Valid even if z is from a different inheritance hierarchy
```

Los genéricos se comparan mediante sus estructuras según el tipo resultante tras aplicar el parámetro genérico; únicamente se compara el resultado final como tipo no genérico.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Invalid as the type argument is used in the final structure

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Valid as the type argument is not used in the final structure
```

Cuando no se especifica el argumento de tipo de los genéricos, todos los argumentos sin especificar se tratan como tipos “any”:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Valid
```

Recuerda:

```
let a: number = 1;
let b: number = 2;
a = b; // Valid, everything is assignable to itself

let c: any;
```

```
c = 1; // Valid, all types are assignable to any

let d: unknown;
d = 1; // Valid, all types are assignable to unknown

let e: unknown;
let e1: unknown = e; // Valid, unknown is only assignable to
itself and any
let e2: any = e; // Valid
let e3: number = e; // Invalid

let f: never;
f = 1; // Invalid, nothing is assignable to never

let g: void;
let g1: any;
g = 1; // Invalid, void is not assignable to or from anything
except any
g = g1; // Valid
```

Ten en cuenta que, cuando “strictNullChecks” está activado, “null” y “undefined” se tratan de forma similar a “void”; de lo contrario, se comportan de forma parecida a “never”.

Los tipos como conjuntos

En TypeScript, un tipo es un conjunto de valores posibles. Este conjunto también se denomina dominio del tipo. Cada valor de un tipo puede considerarse un elemento de un conjunto. Un tipo establece las restricciones que debe satisfacer cada elemento para considerarse miembro del conjunto. La tarea principal de TypeScript consiste en comprobar y verificar si un conjunto es subconjunto de otro.

TypeScript admite distintos tipos de conjuntos:

Término de conjuntos	TypeScript	Notas
Conjunto vacío	never	“never” no contiene nada aparte de sí mismo
Conjunto de un elemento	undefined / null / tipo literal	
Conjunto finito	boolean / unión	
Conjunto infinito	string / number / object	
Conjunto universal	any / unknown	Todo elemento pertenece a “any” y todo conjunto es subconjunto suyo / “unknown” es la alternativa segura de “any”

Estos son algunos ejemplos:

TypeScript	Término de conjuntos	Ejemplo
never	∅ (conjunto vacío)	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
Tipo literal	Conjunto de un elemento	type X = ‘X’;
		type Y = 7;
Valor asignable a T	Valor ∈ T (pertenece a)	type XY = ‘X’ | ‘Y’;
		const x: XY = ‘X’;
T1 asignable a T2	T1 ⊆ T2 (subconjunto de)	type XY = ‘X’ | ‘Y’;
		const x: XY = ‘X’;
		const j: XY = ‘J’; // Type ‘“J”’ is not assignable to type ‘XY’.

T1 extends T2	T1 ⊆ T2 (subconjunto de)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (unión)	type XY = ‘X’ | ‘Y’; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (intersección)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: ‘a’, b: ‘b’ }
unknown	Conjunto universal	const x: unknown = 1

Una unión, (T1 | T2), crea un conjunto más amplio (ambos):

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Valid
```

Una intersección, (T1 & T2), crea un conjunto más restringido (solo lo compartido):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
```

```
const r: XY = { a: 'a' }; // Invalid
const j: XY = { a: 'a', b: 'b' }; // Valid
```

En este contexto, la palabra clave `extends` puede entenderse como «subconjunto de». Establece una restricción para un tipo. Cuando `extends` se utiliza con un genérico, restringe el parámetro de tipo genérico a un tipo más específico.

Ten en cuenta que aquí `extends` no tiene relación con la herencia de clases en el sentido de la programación orientada a objetos.

TypeScript trabaja con tipos estructurales y no posee una jerarquía nominal estricta. De hecho, como muestra el siguiente ejemplo, dos tipos pueden solaparse sin que ninguno sea subtipo del otro, porque TypeScript tiene en cuenta la estructura o forma de los objetos.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
```

```
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Valid
```

Asignar un tipo: declaraciones y aserciones de tipo

En TypeScript se puede asignar un tipo de distintas formas:

Declaración de tipo

En el siguiente ejemplo utilizamos x: X (“: Type”) para declarar un tipo para la variable x.

```
type X = {
    a: string;
};

// Type declaration
const x: X = {
    a: 'a',
};
```

Si la variable no tiene el formato especificado, TypeScript notificará un error. Por ejemplo:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
```

```
    b: 'b', // Error: Object literal may only specify known
properties
};
```

Aserción de tipo

Es posible añadir una aserción mediante la palabra clave `as`. Esto indica al compilador que el desarrollador dispone de más información sobre un tipo y silencia los posibles errores.

Por ejemplo:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

En el ejemplo anterior se afirma mediante la palabra clave as que el objeto x tiene el tipo X. Esto informa al compilador de TypeScript de que el objeto se ajusta al tipo especificado, aunque tenga una propiedad adicional b que no aparece en la definición del tipo.

Las aserciones de tipo resultan útiles cuando es necesario especificar un tipo más concreto, especialmente al trabajar con el DOM. Por ejemplo:

```
const myInput = document.getElementById('my_input') as
HTMLInputElement;
```

Aquí, la aserción de tipo as HTMLInputElement indica a TypeScript que el resultado de getElementById debe tratarse como un HTMLInputElement. Las aserciones de tipo también

pueden utilizarse para reasignar claves, como muestra el siguiente ejemplo con literales de plantilla:

```
type J<Type> = {
    [Property in keyof Type as `prefix_${string &
        Property}`]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

En este ejemplo, el tipo `J<Type>` utiliza un tipo mapeado con un literal de plantilla para reasignar las claves de Type. Crea propiedades nuevas añadiendo “prefix_” a cada clave, cuyos valores correspondientes son funciones que devuelven los valores de las propiedades originales.

Conviene señalar que TypeScript no ejecuta la comprobación de propiedades adicionales cuando se utiliza una aserción de tipo. Por tanto, suele ser preferible utilizar una declaración de tipo cuando se conoce de antemano la estructura del objeto.

Declaraciones de ambiente

Las declaraciones de ambiente son archivos que describen tipos para código JavaScript y cuyos nombres tienen el formato `.d.ts`. Normalmente se importan y utilizan para anotar bibliotecas JavaScript existentes o añadir tipos a archivos JS existentes del proyecto.

Los tipos de muchas bibliotecas habituales pueden encontrarse en: https://github.com/DefinitelyTyped/DefinitelyTyped/

y pueden instalarse mediante:

```
npm install --save-dev @types/library-name
```

Para tus propias declaraciones de ambiente, puedes importar mediante una referencia de «triple barra»:

```
/// <reference path="./library-types.d.ts" />
```

Puedes utilizar declaraciones de ambiente incluso dentro de archivos JavaScript mediante `// @ts-check`.

La palabra clave `declare` permite definir tipos para código JavaScript existente sin importarlo y actúa como marcador de posición para tipos procedentes de otro archivo o del ámbito global.

Comprobación de propiedades y de propiedades adicionales

TypeScript se basa en un sistema de tipos estructural, pero la comprobación de propiedades adicionales permite verificar si un objeto tiene exactamente las propiedades especificadas en el tipo.

La comprobación de propiedades adicionales se realiza, por ejemplo, al asignar literales de objeto a variables o pasarlos como argumentos a funciones.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
```

```
const x: X = y; // Valid because structural typing
const w: X = { a: 'a', b: 'b' }; // Invalid because excess
property checking
```

Tipos débiles

Un tipo se considera débil cuando solo contiene un conjunto de propiedades opcionales:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript considera erróneo asignar cualquier valor a un tipo débil cuando no existe solapamiento. Por ejemplo, lo siguiente genera un error:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Invalid
```

Aunque no se recomienda, si es necesario puede omitirse esta comprobación mediante una aserción de tipo:

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Valid
```

O añadiendo unknown a la firma de índice del tipo débil:

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Valid
```

Comprobación estricta de literales de objeto (frescura)

La comprobación estricta de literales de objeto, a veces denominada «frescura», es una característica de TypeScript que ayuda a detectar propiedades adicionales o mal escritas que pasarían inadvertidas en las comprobaciones normales de tipos estructurales.

Al crear un literal de objeto, el compilador de TypeScript lo considera «fresco». Si se asigna a una variable o se pasa como parámetro, TypeScript generará un error cuando el literal especifique propiedades inexistentes en el tipo de destino.

Sin embargo, la «frescura» desaparece cuando se amplía un literal de objeto o se utiliza una aserción de tipo.

Estos son algunos ejemplos:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Freshness check: Invalid assignment
var y: Y;
```

```
y = { a: 'a', bx: 'bx' }; // Freshness check: Invalid assignment

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Widening: No errors, structurally type compatible

fn({ a: 'a', bx: 'b' }); // Freshness check: Invalid argument

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Widening: No Freshness check
```

Inferencia de tipos

TypeScript puede inferir tipos cuando no se proporciona ninguna anotación durante:

- La inicialización de variables.
- La inicialización de miembros.
- La asignación de valores predeterminados a parámetros.
- La determinación del tipo de retorno de una función.

Por ejemplo:

```
let x = 'x'; // The type inferred is string
```

El compilador de TypeScript analiza el valor o la expresión y determina su tipo a partir de la información disponible.

Inferencias más avanzadas

Cuando se utilizan varias expresiones en la inferencia de tipos, TypeScript busca los «mejores tipos comunes». Por ejemplo:

```
let x = [1, 'x', 1, null]; // The type inferred is: (string | number | null)[]
```

Si el compilador no encuentra los mejores tipos comunes, devuelve un tipo de unión. Por ejemplo:

```
let x = [new RegExp('x'), new Date()]; // Type inferred is: (RegExp | Date)[]
```

TypeScript utiliza el «tipado contextual», basado en la ubicación de la variable, para inferir tipos. En el siguiente ejemplo, el compilador sabe que `e` es de tipo `MouseEvent` por el tipo del evento `click` definido en el archivo lib.d.ts, que contiene declaraciones de ambiente para varias construcciones habituales de JavaScript y el DOM:

```
window.addEventListener('click', function (e) {}); // The inferred type of e is MouseEvent
```

Ampliación de tipos

La ampliación de tipos es el proceso mediante el cual TypeScript asigna un tipo a una variable inicializada sin una anotación de tipo. Permite pasar de tipos restringidos a tipos más amplios, pero no a la inversa. En el siguiente ejemplo:

```
let x = 'x'; // TypeScript infers as string, a wide type
let y: 'y' | 'x' = 'y'; // y types is a union of literal types
y = x; // Invalid Type 'string' is not assignable to type '"x" | "y"'.
```

TypeScript asigna `string` a `x` a partir del único valor proporcionado durante la inicialización (`x`); este es un ejemplo de ampliación.

TypeScript ofrece formas de controlar el proceso de ampliación, por ejemplo mediante “const”.

Const

Utilizar la palabra clave `const` al declarar una variable produce en TypeScript una inferencia de tipo más restringida.

Por ejemplo:

```
const x = 'x'; // TypeScript infers the type of x as 'x', a narrower type
let y: 'y' | 'x' = 'y';
y = x; // Valid: The type of x is inferred as 'x'
```

Al utilizar `const` para declarar la variable x, su tipo se restringe al valor literal concreto ‘x’. Como el tipo de x está restringido, puede asignarse a la variable y sin errores. El tipo puede inferirse así porque las variables `const` no pueden reasignarse, por lo que su tipo puede restringirse a un tipo literal concreto; en este caso, el tipo literal ‘x’.

Modificador const en parámetros de tipo

Desde TypeScript 5.0 es posible especificar el atributo `const` en un parámetro de tipo genérico. Esto permite inferir el tipo más preciso posible. Veamos un ejemplo sin utilizar `const`:

```
function identity<T>(value: T) {
    // No const here
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred is: { a: string; b: string; }
```

Como puedes ver, las propiedades `a` y `b` se infieren con el tipo `string`.

Veamos ahora la diferencia con la versión que utiliza `const`:

```
function identity<const T>(value: T) {
    // Using const modifier on type parameters
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred
is: { a: "a"; b: "b"; }
```

Ahora podemos ver que las propiedades `a` y `b` se infieren como literales de cadena y no simplemente como tipos `string`.

Aserción const

Esta característica permite declarar una variable con un tipo literal más preciso según su valor de inicialización, indicando al compilador que debe tratar el valor como un literal inmutable. Estos son algunos ejemplos:

En una sola propiedad:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

En un objeto completo:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Esto puede resultar especialmente útil al definir el tipo de una tupla:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple of readonly [1, 2, 3]
```

Anotación de tipo explícita

Podemos ser explícitos e indicar un tipo. En el siguiente ejemplo, la propiedad `x` es de tipo `number`:

```
const v = {
    x: 1, // Inferred type: number (widening)
};
v.x = 3; // Valid
```

Podemos hacer que la anotación de tipo sea más específica mediante una unión de tipos literales:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x is now a union of literal types: 1 | 2 | 3
};
v.x = 3; // Valid
v.x = 100; // Invalid
```

Restricción de tipos

La restricción de tipos es el proceso de TypeScript mediante el cual un tipo general se reduce a otro más específico. Ocurre cuando TypeScript analiza el código y determina que ciertas condiciones u operaciones pueden precisar la información del tipo.

Los tipos pueden restringirse de distintas formas, entre ellas:

Condiciones

Mediante sentencias condicionales como `if` o `switch`, TypeScript puede restringir el tipo según el resultado de la condición. Por ejemplo:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // The type is number, which had been narrowed by
the condition
}
```

Lanzar una excepción o devolver

Lanzar un error o devolver anticipadamente desde una rama puede ayudar a TypeScript a restringir un tipo. Por ejemplo:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Otras formas de restringir tipos en TypeScript son:

- Operador `instanceof`: se utiliza para comprobar si un objeto es instancia de una clase concreta.
- Operador `in`: se utiliza para comprobar si una propiedad existe en un objeto.
- Operador `typeof`: se utiliza para comprobar el tipo de un valor en tiempo de ejecución.
- Funciones integradas como `Array.isArray()`: se utilizan para comprobar si un valor es un array.

Unión discriminada

Una «unión discriminada» es un patrón de TypeScript en el que se añade una «etiqueta» explícita a los objetos para distinguir entre distintos tipos de una unión. Este patrón también se denomina «unión etiquetada». En el siguiente ejemplo, la propiedad “type” representa la etiqueta:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // type is A
        case 'type_b':
            return input.value + 'extra'; // type is B
    }
};
```

Guardas de tipo definidas por el usuario

Cuando TypeScript no puede determinar un tipo, es posible escribir una función auxiliar denominada «guarda de tipo definida por el usuario». En el siguiente ejemplo utilizaremos un predicado de tipo para restringir el tipo después de aplicar un filtro:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // The type is (string
| null)[], TypeScript was not able to infer the type properly

const isValid = (item: string | null): item is string => item
!== null; // Custom type guard
```

```
const r2 = data.filter(isValid); // The type is fine now string[], by using the predicate type guard we were able to narrow the type
```

Restricción mediante switch-true

TypeScript 5.3 añade la restricción mediante switch-true, que permite sustituir cadenas if/else complejas por switch (true) con condiciones booleanas. Mejora la legibilidad y sigue restringiendo los tipos. Es similar a la coincidencia de patrones, pero más sencilla.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Tipos primitivos

TypeScript admite 7 tipos primitivos. Un tipo de datos primitivo es un tipo que no es un objeto y no tiene métodos asociados. En TypeScript, todos los tipos primitivos son inmutables, lo que significa que sus valores no pueden cambiar una vez asignados.

string

El tipo primitivo `string` almacena datos de texto y su valor siempre se escribe entre comillas dobles o simples.

```
const x: string = 'x';
const y: string = 'y';
```

Las cadenas pueden ocupar varias líneas si están delimitadas por el carácter de acento grave (`):

```
let sentence: string = `xxx,
   yyy`;
```

boolean

El tipo de datos `boolean` de TypeScript almacena un valor binario: `true` o `false`.

```
const isReady: boolean = true;
```

number

Un tipo de datos `number` de TypeScript se representa mediante un valor de coma flotante de 64 bits. Un tipo `number` puede representar enteros y fracciones. TypeScript también admite valores hexadecimales, binarios y octales. Por ejemplo:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimal starts with
0x
const binary: number = 0b1010; // Binary starts with 0b
const octal: number = 0o633; // Octal starts with 0o
```

bigint

Un `bigint` representa valores enteros que pueden superar el entero seguro máximo admitido por `number`, que es 2^53 - 1.

Puede crearse un `bigint` llamando a la función integrada `BigInt()` o añadiendo `n` al final de cualquier literal numérico entero:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Notas:

- Los valores `bigint` no pueden mezclarse con `number`; para operar juntos, ambos valores deben convertirse al mismo tipo. Tampoco pueden utilizarse con el objeto integrado `Math`.
- Los valores `bigint` solo están disponibles si la configuración de destino es ES2020 o superior.

Symbol

Los símbolos son identificadores únicos que pueden utilizarse como claves de propiedades en objetos para evitar conflictos de nombres.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null y undefined

Los tipos `null` y `undefined` representan la ausencia de un valor.

El tipo `undefined` significa que el valor no se ha asignado o inicializado, o indica una ausencia involuntaria de valor.

El tipo `null` significa que sabemos que el campo no tiene valor y, por tanto, el valor no está disponible; indica una ausencia intencionada.

Array

Un `array` es un tipo de datos que puede almacenar varios valores, sean o no del mismo tipo. Puede definirse con la siguiente sintaxis:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript admite arrays de solo lectura mediante la siguiente sintaxis:

```
const x: readonly string[] = ['a', 'b']; // Readonly modifier
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Invalid
```

TypeScript admite tuplas y tuplas de solo lectura:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

El tipo de datos `any` representa literalmente «cualquier» valor y es el predeterminado cuando TypeScript no puede inferir el tipo o este no se especifica.

Al utilizar `any`, el compilador de TypeScript omite la comprobación de tipos, por lo que no hay seguridad de tipos cuando se utiliza `any`. En general, no utilices `any` para silenciar el compilador cuando se produzca un error; céntrate en corregirlo, ya que `any` permite incumplir contratos y perder las ventajas de la finalización automática de TypeScript.

El tipo `any` puede resultar útil durante una migración gradual de JavaScript a TypeScript, ya que permite silenciar el compilador.

En proyectos nuevos, utiliza la configuración `noImplicitAny` de TypeScript, que hace que TypeScript emita errores cuando se utiliza o infiere `any`.

El tipo `any` suele ser una fuente de errores que puede ocultar problemas reales con los tipos. Evita utilizarlo siempre que sea posible.

Anotaciones de tipo

En las variables declaradas mediante `var`, `let` y `const` puede añadirse un tipo de forma opcional:

```
const x: number = 1;
```

TypeScript infiere bien los tipos, especialmente los sencillos, por lo que estas declaraciones no son necesarias en la mayoría de los casos.

En las funciones pueden añadirse anotaciones de tipo a los parámetros:

```
function sum(a: number, b: number) {
    return a + b;
}
```

El siguiente ejemplo utiliza una función anónima (también denominada función lambda):

```
const sum = (a: number, b: number) => a + b;
```

Estas anotaciones pueden omitirse cuando un parámetro tiene un valor predeterminado:

```
const sum = (a = 10, b: number) => a + b;
```

También pueden añadirse anotaciones del tipo de retorno a las funciones:

```
const sum = (a = 10, b: number): number => a + b;
```

Esto resulta especialmente útil en funciones más complejas, ya que escribir el tipo de retorno antes de la implementación puede ayudarte a razonar sobre la función.

En general, considera anotar las firmas de tipo, pero no las variables locales del cuerpo, y añade siempre tipos a los literales de objeto.

Propiedades opcionales

Un objeto puede especificar propiedades opcionales añadiendo un signo de interrogación `?` al final del nombre de la propiedad:

```
type X = {
    a: number;
    b?: number; // Optional
};
```

Es posible especificar un valor predeterminado cuando una propiedad es opcional:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Propiedades de solo lectura

Es posible impedir la escritura en una propiedad mediante el modificador `readonly`, que garantiza que la propiedad no pueda reescribirse, pero no proporciona ninguna garantía de inmutabilidad total:

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Firmas de índice

En TypeScript podemos utilizar `string`, `number` y `symbol` como firmas de índice:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Same result as k[1]
```

Ten en cuenta que JavaScript convierte automáticamente un índice `number` en un índice `string`, por lo que `k[1]` y `k["1"]` devuelven el mismo valor.

Extensión de tipos

Es posible extender una `interface` (copiar miembros de otro tipo):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

También es posible extender varios tipos:

```
interface A {
    a: string;
}
interface B {
    b: string;
```

```
}
interface Y extends A, B {
    y: string;
}
```

La palabra clave `extends` solo funciona con interfaces y clases; para los tipos debe utilizarse una intersección:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Es posible extender un tipo mediante una interfaz, pero no a la inversa:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Tipos literales

Un tipo literal es un conjunto de un solo elemento dentro de un tipo colectivo; define un valor muy concreto que es un primitivo de JavaScript.

Los tipos literales de TypeScript son números, cadenas y booleanos.

Ejemplos de literales:

```
const a = 'a'; // String literal type
const b = 1; // Numeric literal type
const c = true; // Boolean literal type
```

Los tipos literales de cadena, numéricos y booleanos se utilizan en uniones, guardas de tipo y alias de tipo. En el siguiente ejemplo puedes ver un alias de tipo de unión. `O` consta únicamente de los valores especificados; ninguna otra cadena es válida:

```
type O = 'a' | 'b' | 'c';
```

Inferencia de literales

La inferencia de literales es una característica de TypeScript que permite inferir el tipo de una variable o parámetro a partir de su valor.

En el siguiente ejemplo vemos que TypeScript considera que `x` es un tipo literal porque su valor no puede cambiar posteriormente, mientras que `y` se infiere como string porque puede modificarse.

```
const x = 'x'; // Literal type of 'x', because this value cannot be changed
let y = 'y'; // Type string, as we can change this value
```

En el siguiente ejemplo vemos que `o.x` se infiere como `string` (y no como el literal `a`), ya que TypeScript considera que su valor puede cambiar posteriormente.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // This is a wider string
```

```
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x));  // Argument of type 'string' is not
assignable to parameter of type 'X'
```

Como puedes ver, el código genera un error al pasar `o.x` a `fn`, ya que X es un tipo más restringido.

Podemos resolver este problema mediante una aserción de tipo con `const` o con el tipo `X`:

```
let o = {
    x: 'a' as const,
};
```

o:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` es una opción del compilador de TypeScript que aplica una comprobación estricta de valores nulos. Cuando está activada, solo se puede asignar `null` o `undefined` a variables y parámetros declarados explícitamente con ese tipo mediante la unión `null | undefined`. Si una variable o parámetro no se declara explícitamente como anulable, TypeScript genera un error para evitar posibles errores en tiempo de ejecución.

Enums

En TypeScript, un `enum` es un conjunto de valores constantes con nombre.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Los enums pueden definirse de distintas formas:

Enums numéricos

En TypeScript, un enum numérico es aquel en el que se asigna un valor numérico a cada constante, comenzando de forma predeterminada por 0.

```
enum Size {
    Small, // value starts from 0
    Medium,
    Large,
}
```

Es posible especificar valores personalizados asignándolos explícitamente:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Enums de cadena

En TypeScript, un enum de cadena es aquel en el que se asigna un valor de cadena a cada constante.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Nota: TypeScript permite utilizar enums heterogéneos en los que coexisten miembros de cadena y numéricos.

Enums constantes

Un enum constante de TypeScript es un tipo especial de enum cuyos valores se conocen durante la compilación y se insertan allí donde se utiliza el enum, lo que produce un código más eficiente.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Se compilará como:

```
console.log('EN' /* Language.English */);
```

Nota: Los enums constantes tienen valores codificados directamente y eliminan el enum, lo que puede resultar más eficiente en bibliotecas autocontenidas, aunque por lo general no es deseable. Además, los enums constantes no pueden tener miembros calculados.

Mapeo inverso

En TypeScript, el mapeo inverso de los enums permite obtener el nombre de un miembro a partir de su valor. De forma predeterminada, los miembros tienen un mapeo directo del nombre al valor, pero pueden crearse mapeos inversos asignando explícitamente valores a cada miembro. Resultan útiles para buscar un miembro por su valor o recorrer todos los miembros. Solo los miembros numéricos generan mapeos inversos; los miembros de cadena no los generan.

El siguiente enum:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Se compila como:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Por tanto, el mapeo de valores a claves funciona con miembros numéricos, pero no con miembros de cadena:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
```

```
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]); // Element implicitly has an
'any' type because index expression is not of type 'number'.
```

Enums de ambiente

Un enum de ambiente de TypeScript se define en un archivo de declaración (*.d.ts) sin una implementación asociada. Permite definir constantes con nombre que pueden utilizarse de forma segura entre distintos archivos sin importar en cada uno los detalles de la implementación.

Miembros calculados y constantes

En TypeScript, un miembro calculado de un enum tiene un valor calculado en tiempo de ejecución, mientras que el valor de un miembro constante se establece durante la compilación y no puede cambiar en tiempo de ejecución. Los miembros calculados se permiten en enums normales, mientras que los miembros constantes se permiten tanto en enums normales como en enums constantes.

```
// Constant members
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 generation at compilation time
```

```
// Computed members
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // random number generated at run time
```

Los enums se representan mediante uniones de los tipos de sus miembros. El valor de cada miembro puede determinarse mediante expresiones constantes o no constantes; a los miembros con valores constantes se les asignan tipos literales. Considera la declaración del tipo E y sus subtipos E.A, E.B y E.C. En este caso, E representa la unión E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Numeric literal
    B = 'bar', // String literal
    C = identity(42), // Opaque computed
}

console.log(E.C); //42
```

Restricción

La restricción de TypeScript es el proceso de precisar el tipo de una variable dentro de un bloque condicional. Resulta útil al trabajar con tipos de unión, donde una variable puede tener más de un tipo.

TypeScript reconoce varias formas de restringir el tipo:

Guardas de tipo typeof

La guarda de tipo typeof es una guarda específica de TypeScript que comprueba el tipo de una variable a partir de su tipo integrado de JavaScript.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x is number
    }
    return -1;
};
```

Restricción por veracidad

La restricción por veracidad de TypeScript comprueba si una variable se evalúa como verdadera o falsa para restringir su tipo en consecuencia.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Restricción por igualdad

La restricción por igualdad de TypeScript comprueba si una variable es igual o no a un valor concreto para restringir su tipo en consecuencia.

Se utiliza junto con sentencias `switch` y operadores de igualdad como `===`, `!==`, `==` y `!=` para restringir tipos.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
```

```
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

Restricción mediante el operador in

La restricción mediante el operador `in` permite restringir el tipo de una variable según exista o no una propiedad en dicho tipo.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

Restricción mediante instanceof

La restricción mediante el operador `instanceof` permite restringir el tipo de una variable según su función constructora, comprobando si un objeto es instancia de una clase o interfaz determinada.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Asignaciones

La restricción mediante asignaciones permite restringir el tipo de una variable según el valor que se le asigne. Cuando se asigna un valor, TypeScript infiere su tipo y restringe el tipo de la variable para que coincida con él.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Análisis del flujo de control

El análisis del flujo de control de TypeScript analiza estáticamente el flujo del código para inferir los tipos de las variables, lo que permite al compilador restringirlos cuando sea necesario según los resultados.

Antes de TypeScript 4.4, el análisis del flujo solo se aplicaba al código dentro de una sentencia if; desde TypeScript 4.4 también puede aplicarse a expresiones condicionales y accesos a propiedades discriminantes referenciados indirectamente mediante variables const.

Por ejemplo:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Algunos ejemplos en los que no se produce la restricción:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // Error, no narrowing because isString it is
not const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // Error, no narrowing because obj is assigned
in function body
    }
};
```

Notas: en las expresiones condicionales se analizan hasta cinco niveles de indirección.

Predicados de tipo

Los predicados de tipo de TypeScript son funciones que devuelven un valor booleano y se utilizan para restringir el tipo de una variable a otro más específico.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
```

```
    }
};
```

TypeScript 5.5 infiere automáticamente predicados de tipo (como `x is T`) en funciones como `.filter`, por lo que sabe cuándo se eliminan valores como undefined y ofrece tipos más precisos y menos errores; funciona con comprobaciones claras (por ejemplo, `x !== undefined`), pero no con otras ambiguas como `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

Uniones discriminadas

Las uniones discriminadas de TypeScript son tipos de unión que utilizan una propiedad común, denominada discriminante, para restringir el conjunto de tipos posibles.

```
type Square = {
    kind: 'square'; // Discriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Discriminant
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
```

```
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

El tipo never

Cuando una variable se restringe a un tipo que no puede contener ningún valor, el compilador de TypeScript infiere que debe ser de tipo `never`, ya que este representa un valor que nunca puede producirse.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
            // val has type never here because it can never be
anything other than a string or a number
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Comprobación exhaustiva

La comprobación exhaustiva garantiza que todos los casos posibles de una unión discriminada se gestionen en una sentencia `switch` o `if`.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
                console.log(exhaustiveCheck); // This line will
never be executed
    }
};
```

El tipo `never` garantiza que el caso predeterminado sea exhaustivo y que TypeScript genere un error si se añade un nuevo valor al tipo Direction sin gestionarlo en la sentencia switch.

Tipos de objeto

En TypeScript, los tipos de objeto describen la forma de un objeto. Especifican los nombres y tipos de sus propiedades, así como si son obligatorias u opcionales.

En TypeScript puedes definir tipos de objeto de dos formas principales:

Una interfaz define la forma de un objeto especificando los nombres, tipos y carácter opcional de sus propiedades.

```
interface User {
    name: string;
```

```
    age: number;
    email?: string;
}
```

Un alias de tipo, al igual que una interfaz, define la forma de un objeto. Sin embargo, también puede crear un tipo personalizado nuevo a partir de un tipo existente o de una combinación de tipos. Esto incluye tipos de unión, tipos de intersección y otros tipos complejos.

```
type Point = {
    x: number;
    y: number;
};
```

También es posible definir un tipo de forma anónima:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Tipo de tupla (anónima)

Un tipo de tupla representa un array con un número fijo de elementos y sus tipos correspondientes. Impone una cantidad concreta de elementos y sus respectivos tipos en un orden fijo. Resulta útil para representar una colección de valores con tipos específicos donde la posición de cada elemento tiene un significado concreto.

```
type Point = [number, number];
```

Tipo de tupla con nombres (etiquetada)

Los tipos de tupla pueden incluir etiquetas o nombres opcionales para cada elemento. Estas etiquetas mejoran la

legibilidad y la asistencia de las herramientas, pero no afectan a las operaciones que pueden realizarse.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Named Tuple plus Anonymous Tuple
```

Tupla de longitud fija

Una tupla de longitud fija impone un número fijo de elementos de tipos concretos e impide modificar su longitud una vez definida.

Las tuplas de longitud fija resultan útiles para representar una colección con una cantidad y unos tipos de elementos concretos, garantizando que la longitud y los tipos no se cambien accidentalmente.

```
const x = [10, 'hello'] as const;
x.push(2); // Error
```

Tipo de unión

Un tipo de unión representa un valor que puede pertenecer a varios tipos. Se indica mediante el símbolo `|` entre cada tipo posible.

```
let x: string | number;
x = 'hello'; // Valid
x = 123; // Valid
```

Tipos de intersección

Un tipo de intersección representa un valor que posee todas las propiedades de dos o más tipos. Se indica mediante el símbolo `&` entre cada tipo.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Intersection

const j: J = {
    a: 'a',
    b: 'b',
};
```

Indexación de tipos

La indexación de tipos permite definir tipos que pueden indexarse mediante una clave desconocida de antemano, utilizando una firma de índice para especificar el tipo de las propiedades no declaradas explícitamente.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Returns a
```

Tipo a partir de un valor

El tipo a partir de un valor hace referencia a la inferencia automática de un tipo desde un valor o expresión.

```
const x = 'x'; // TypeScript infers 'x' as a string literal with 'const' (immutable), but widens it to 'string' with 'let' (reassignable).
```

Tipo a partir del retorno de una función

El tipo a partir del retorno de una función permite inferir automáticamente el tipo de retorno según su implementación. De este modo, TypeScript puede determinar el tipo del valor devuelto sin anotaciones de tipo explícitas.

```
const add = (x: number, y: number) => x + y; // TypeScript can infer that the return type of the function is a number
```

Tipo a partir de un módulo

El tipo a partir de un módulo permite utilizar los valores exportados para inferir automáticamente sus tipos. Cuando un módulo exporta un valor con un tipo concreto, TypeScript puede usar esa información al importarlo en otro módulo.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r is number
```

Tipos mapeados

Los tipos mapeados de TypeScript permiten crear tipos nuevos a partir de otro existente transformando cada propiedad

mediante una función de mapeo. Así pueden representar la misma información con un formato distinto. Para crear uno, se accede a las propiedades de un tipo existente mediante el operador `keyof` y después se modifican para producir un tipo nuevo. En el siguiente ejemplo:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

definimos MyMappedType para recorrer las propiedades de T y crear un tipo nuevo donde cada propiedad es un array de su tipo original. Con él creamos MyNewType, que representa la misma información que MyType, pero con cada propiedad como array.

Modificadores de tipos mapeados

Los modificadores de tipos mapeados permiten transformar las propiedades de un tipo existente:

- `readonly` o `+readonly`: convierte una propiedad del tipo mapeado en una propiedad de solo lectura.
- `-readonly`: permite que una propiedad del tipo mapeado sea mutable.

- `?`: designa una propiedad del tipo mapeado como opcional.

Ejemplos:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // All properties marked as read-only

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // All properties marked as mutable

type MyPartial<T> = { [P in keyof T]?: T[P] }; // All properties marked as optional
```

Tipos condicionales

Los tipos condicionales permiten crear un tipo que depende de una condición, cuyo resultado determina el tipo que se creará. Se definen mediante la palabra clave `extends` y un operador ternario para elegir entre dos tipos.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Type true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Type false
```

Tipos condicionales distributivos

Los tipos condicionales distributivos permiten distribuir un tipo sobre una unión aplicando una transformación a cada miembro por separado. Esto resulta especialmente útil con tipos mapeados o tipos de orden superior.

```ts
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number |
boolean | null
```

Inferencia de tipos con infer en tipos condicionales

La palabra clave `infer` se utiliza en tipos condicionales para inferir (extraer) el tipo de un parámetro genérico desde un tipo que depende de él. Esto permite escribir definiciones más flexibles y reutilizables.

```ts
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Tipos condicionales predefinidos

En TypeScript, los tipos condicionales predefinidos son tipos integrados que proporciona el lenguaje para realizar transformaciones habituales según las características de un tipo.

`Exclude<UnionType, ExcludedType>`: elimina de Type todos los tipos asignables a ExcludedType.

`Extract<Type, Union>`: extrae de Union todos los tipos asignables a Type.

`NonNullable<Type>`: elimina null y undefined de Type.

`ReturnType<Type>`: extrae el tipo de retorno de una función Type.

`Parameters<Type>`: extrae los tipos de los parámetros de una función Type.

`Required<Type>`: convierte en obligatorias todas las propiedades de Type.

`Partial<Type>`: convierte en opcionales todas las propiedades de Type.

`Readonly<Type>`: convierte todas las propiedades de Type en propiedades de solo lectura.

Tipos de unión de plantilla

Los tipos de unión de plantilla pueden utilizarse para combinar y manipular texto dentro del sistema de tipos. Por ejemplo:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Tipo any

El tipo `any` es un tipo especial (supertipo universal) que puede representar cualquier clase de valor: primitivos, objetos, arrays, funciones, errores o símbolos. Suele utilizarse cuando el tipo de un valor no se conoce durante la compilación o al trabajar con API o bibliotecas externas sin tipos de TypeScript.

Al utilizar `any` indicas al compilador de TypeScript que los valores deben representarse sin limitaciones. Para maximizar la seguridad de tipos, ten en cuenta lo siguiente:

- Limita el uso de any a casos concretos en los que el tipo sea realmente desconocido.
- No devuelvas tipos any desde una función, ya que debilita la seguridad de tipos del código que la utiliza.
- En lugar de any, utiliza `@ts-ignore` si necesitas silenciar el compilador.

```
let value: any;
value = true; // Valid
value = 7; // Valid
```

Tipo unknown

En TypeScript, unknown representa un valor de tipo desconocido. A diferencia de any, exige una comprobación o aserción antes de utilizarlo de una forma concreta; no se permite ninguna operación sobre un unknown sin afirmar o restringir primero ese unknown a un tipo más específico.

El tipo unknown solo puede asignarse a any y al propio unknown, y es una alternativa segura a any.

```
let value: unknown;

let value1: unknown = value; // Valid
let value2: any = value; // Valid
let value3: boolean = value; // Invalid
let value4: number = value; // Invalid

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Tipo void

El tipo `void` indica que una función no devuelve ningún valor.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Tipo never

El tipo `never` representa valores que nunca se producen. Se utiliza para indicar funciones o expresiones que nunca devuelven un valor o que lanzan un error.

Por ejemplo, un bucle infinito:

```
const infiniteLoop = (): never => {
    while (true) {
        // do something
    }
};
```

Lanzar un error:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

El tipo `never` ayuda a garantizar la seguridad de tipos y detectar posibles errores. Permite que TypeScript analice e infiera tipos más precisos al combinarlo con otros tipos y sentencias de flujo de control. Por ejemplo:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
```

```
        case 'up':
            // move up
            break;
        case 'down':
            // move down
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
    }
};
```

Interfaz y tipo

Sintaxis habitual

En TypeScript, las interfaces definen la estructura de los objetos y especifican los nombres y tipos de las propiedades o métodos que deben tener. La sintaxis habitual para definir una interfaz es la siguiente:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

La definición de tipos utiliza una sintaxis similar:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` o `type TypeName`: define el nombre de la interfaz. `property1: Type1`: especifica las propiedades de la interfaz y sus tipos correspondientes. Pueden definirse varias propiedades separadas por punto y coma. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: especifica los métodos de la interfaz. Se definen con su nombre, una lista de parámetros entre paréntesis y el tipo de retorno. Pueden definirse varios métodos separados por punto y coma.

Ejemplo de interfaz:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Ejemplo de tipo:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

En TypeScript, los tipos definen la forma de los datos y aplican la comprobación de tipos. Existen varias sintaxis habituales según el caso de uso. Estos son algunos ejemplos:

Tipos básicos

```
let myNumber: number = 123; // number type
let myBoolean: boolean = true; // boolean type
let myArray: string[] = ['a', 'b']; // array of strings
let myTuple: [string, number] = ['a', 123]; // tuple
```

Objetos e interfaces

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Tipos de unión e intersección

```
type MyType = string | number; // Union type
let myUnion: MyType = 'hello'; // Can be a string
myUnion = 123; // Or a number

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection type
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Object with both name and age properties
```

Tipos primitivos integrados

TypeScript incluye varios tipos primitivos integrados que pueden utilizarse para definir variables, parámetros de funciones y tipos de retorno:

- `number`: representa valores numéricos, incluidos enteros y números de coma flotante.
- `string`: representa datos de texto.
- `boolean`: representa valores lógicos, que pueden ser true o false.
- `null`: representa la ausencia de un valor.
- `undefined`: representa un valor que no se ha asignado o definido.
- `symbol`: representa un identificador único. Los símbolos suelen utilizarse como claves de propiedades de objetos.
- `bigint`: representa enteros de precisión arbitraria.
- `any`: representa un tipo dinámico o desconocido. Sus variables pueden contener valores de cualquier tipo y

omiten la comprobación de tipos.
- `void`: representa la ausencia de cualquier tipo. Suele utilizarse como tipo de retorno de funciones que no devuelven un valor.
- `never`: representa valores que nunca se producen. Suele utilizarse como tipo de retorno de funciones que lanzan un error o entran en un bucle infinito.

Objetos JS integrados habituales

TypeScript es un superconjunto de JavaScript e incluye todos sus objetos integrados habituales. Puedes encontrar una lista extensa en la documentación de Mozilla Developer Network (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Esta es una lista de algunos objetos JavaScript integrados de uso habitual:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set

- Promise
- Intl

Sobrecargas

Las sobrecargas de funciones de TypeScript permiten definir varias firmas para un mismo nombre, de modo que la función pueda llamarse de distintas formas. Este es un ejemplo:

```
// Overloads
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Valid
sayHi(['aa', 'bb']); // Valid
```

Este es otro ejemplo de sobrecargas de funciones dentro de una `class`:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }
```

```
    // overload
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implementation
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Combinación y extensión

La combinación y la extensión son dos conceptos distintos relacionados con los tipos y las interfaces.

La combinación permite reunir varias declaraciones con el mismo nombre en una única definición; por ejemplo, al definir varias veces una interfaz con el mismo nombre:

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
```

```
    b: 7,
};
```

La extensión permite extender o heredar tipos o interfaces existentes para crear otros nuevos. Es un mecanismo para añadir propiedades o métodos sin modificar la definición original. Ejemplo:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Diferencias entre tipo e interfaz

Combinación de declaraciones (ampliación):

Las interfaces admiten la combinación de declaraciones: pueden definirse varias con el mismo nombre y TypeScript las combina en una sola con todas sus propiedades y métodos. Los tipos, en cambio, no la admiten. Esto resulta útil para añadir funcionalidad o personalizar tipos existentes sin modificar las

definiciones originales, o para corregir tipos ausentes o incorrectos.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Extender otros tipos o interfaces:

Tanto los tipos como las interfaces pueden extender otros tipos o interfaces, aunque la sintaxis es distinta. Las interfaces utilizan `extends` para heredar propiedades y métodos, pero no pueden extender tipos complejos como una unión.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

Con los tipos se utiliza el operador & para combinar varios en uno solo (intersección).

```ts
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Tipos de unión e intersección:

Los tipos son más flexibles al definir uniones e intersecciones. Con `type` pueden crearse fácilmente uniones mediante `|` e intersecciones mediante `&`. Las interfaces también pueden representar uniones indirectamente, pero no disponen de compatibilidad integrada con las intersecciones.

```ts
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Intersection
```

Ejemplo con interfaces:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union of interfaces
```

Clase

Sintaxis habitual de las clases

La palabra clave `class` se utiliza para definir una clase. A continuación se muestra un ejemplo:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

La palabra clave `class` define una clase denominada “Person”.

La clase tiene dos propiedades privadas: name de tipo `string` y age de tipo `number`.

El constructor se define mediante la palabra clave `constructor`. Recibe name y age como parámetros y los asigna a las propiedades correspondientes.

La clase tiene un método `public` denominado sayHi que registra un saludo.

Para crear una instancia de una clase en TypeScript puede utilizarse la palabra clave `new`, seguida del nombre de la clase y de paréntesis `()`. Por ejemplo:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Output: Hello, my name is John Doe and I
am 25 years old.
```

Constructor

Los constructores son métodos especiales de una clase que inicializan las propiedades del objeto al crear una instancia.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}
```

```
const john = new Person('Simon', 17);
john.sayHello();
```

Es posible sobrecargar un constructor mediante la siguiente sintaxis:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

En TypeScript es posible definir varias sobrecargas de constructor, pero solo puede haber una implementación compatible con todas ellas; esto puede lograrse mediante un parámetro opcional.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
```

```
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Constructores privados y protegidos

En TypeScript, los constructores pueden marcarse como privados o protegidos, lo que restringe su accesibilidad y uso.

Constructores privados: Solo pueden llamarse desde la propia clase. Suelen utilizarse para imponer un patrón singleton o restringir la creación de instancias a un método factoría de la clase.

Constructores protegidos: Resultan útiles para crear una clase base que no debe instanciarse directamente, pero que puede ser extendida por subclases.

```
class BaseClass {
    protected constructor() {}
}
```

```
class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Attempting to instantiate the base class directly will result in an error
// const baseObj = new BaseClass(); // Error: Constructor of class 'BaseClass' is protected.

// Create an instance of the derived class
const derivedObj = new DerivedClass(10);
```

Modificadores de acceso

Los modificadores de acceso `private`, `protected` y `public` controlan la visibilidad y accesibilidad de los miembros de una clase, como propiedades y métodos. Son esenciales para aplicar la encapsulación y establecer límites al acceso y modificación del estado interno.

El modificador `private` restringe el acceso al miembro a la clase que lo contiene.

El modificador `protected` permite acceder al miembro desde la clase que lo contiene y sus clases derivadas.

El modificador `public` permite acceder al miembro sin restricciones desde cualquier lugar.

Get y set

Los getters y setters son métodos especiales que permiten definir un comportamiento personalizado de acceso y modificación para las propiedades de una clase. Permiten encapsular el estado interno de un objeto y añadir lógica al obtener o establecer valores. En TypeScript se definen mediante las palabras clave `get` y `set`, respectivamente. Este es un ejemplo:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

Autoaccesores en clases

TypeScript 4.9 añade compatibilidad con los autoaccesores, una característica futura de ECMAScript. Se parecen a las propiedades de clase, pero se declaran mediante la palabra clave “accessor”.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Los autoaccesores se transforman en accesores privados `get` y `set` que operan sobre una propiedad inaccesible.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

En TypeScript, la palabra clave `this` hace referencia a la instancia actual de una clase dentro de sus métodos o constructores. Permite acceder y modificar las propiedades y métodos de la clase desde su propio ámbito. Proporciona una forma de acceder y manipular el estado interno de un objeto desde sus métodos.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}
```

```
const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Propiedades de parámetros

Las propiedades de parámetros permiten declarar e inicializar propiedades de clase directamente en los parámetros del constructor, evitando código repetitivo. Por ejemplo:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
            // The "private" and "public" keywords in the
constructor
              // automatically declare and initialize the
corresponding class properties.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Clases abstractas

Las clases abstractas se utilizan principalmente para la herencia. Permiten definir propiedades y métodos comunes que pueden heredar las subclases. Resultan útiles para definir un comportamiento común y exigir que las subclases implementen determinados métodos. Permiten crear una jerarquía en la que

la clase base abstracta proporciona una interfaz compartida y funcionalidad común.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Output: Whiskers meows.
```

Con genéricos

Las clases con genéricos permiten definir clases reutilizables que funcionan con tipos distintos.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
```

```
    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Decoradores

Los decoradores proporcionan un mecanismo para añadir metadatos, modificar comportamientos, validar o ampliar la funcionalidad del elemento de destino. Son funciones que se ejecutan en tiempo de ejecución y pueden aplicarse varios decoradores a una declaración.

Los decoradores son una característica experimental y los siguientes ejemplos solo son compatibles con TypeScript 5 o superior con ES6.

En versiones anteriores a TypeScript 5 deben activarse mediante la propiedad `experimentalDecorators` de `tsconfig.json` o con `--experimentalDecorators` en la línea de comandos (aunque el siguiente ejemplo no funcionará).

Algunos usos habituales de los decoradores son:

- Observar cambios en propiedades.
- Observar llamadas a métodos.
- Añadir propiedades o métodos.

- Validar en tiempo de ejecución.
- Serializar y deserializar automáticamente.
- Registrar eventos.
- Autorizar y autenticar.
- Proteger frente a errores.

Nota: los decoradores de la versión 5 no permiten decorar parámetros.

Tipos de decoradores:

Decoradores de clase

Los decoradores de clase resultan útiles para extender una clase existente, por ejemplo añadiendo propiedades o métodos, o recopilando instancias. En el siguiente ejemplo añadimos un método `toString` que convierte la clase en una representación de cadena.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
```

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Logs:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Decorador de propiedades

Los decoradores de propiedades sirven para modificar el comportamiento de una propiedad, como sus valores de inicialización. El siguiente código establece que una propiedad esté siempre en mayúsculas:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}
```

```
console.log(new MyClass().prop1); // Logs: HELLO!
```

Decorador de métodos

Los decoradores de métodos permiten cambiar o mejorar el comportamiento de los métodos. A continuación se muestra un registrador sencillo:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

Registra:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Decoradores de getters y setters

Los decoradores de getters y setters permiten cambiar o mejorar el comportamiento de los accesores de clase. Resultan útiles, por ejemplo, para validar asignaciones de propiedades. Este es un ejemplo sencillo de decorador de getter:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
```

```
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Valid: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Throw: Invalid!
```

Metadatos de decoradores

Los metadatos de decoradores simplifican la aplicación y el uso de metadatos en cualquier clase. Los decoradores pueden acceder a una nueva propiedad metadata del objeto de contexto, que puede servir como clave tanto para primitivos como para objetos. La información de metadatos puede consultarse en la clase mediante `Symbol.metadata`.

Los metadatos pueden utilizarse para depuración, serialización o inyección de dependencias mediante decoradores, entre otros fines.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Simple polyfill

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // Context contains property metadata: DecoratorMetadata
```

```
function setMetadata(_target: any, context: Context) {
    // Set the metadata object with a primitive value
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Get metadata
information

console.log(JSON.stringify(metadata));                    //
{"bar":true,"baz":true,"foo":true}
```

Herencia

La herencia es el mecanismo por el que una clase hereda propiedades y métodos de otra, denominada clase base o superclase. La clase derivada, también llamada clase hija o subclase, puede ampliar y especializar su funcionalidad añadiendo propiedades y métodos o sobrescribiendo los existentes.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
```

```
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Create an instance of the base class
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Create an instance of the derived class
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript no admite herencia múltiple en el sentido tradicional, sino que permite heredar de una única clase base. TypeScript admite varias interfaces. Una interfaz puede definir un contrato para la estructura de un objeto y una clase puede implementar varias interfaces, lo que le permite heredar comportamiento y estructura de varias fuentes.

```
interface Flyable {
    fly(): void;
```

```
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

La palabra clave `class` de TypeScript, al igual que en JavaScript, suele considerarse azúcar sintáctico. Se introdujo en ECMAScript 2015 (ES6) para ofrecer una sintaxis más familiar al crear y utilizar objetos basados en clases. Sin embargo, TypeScript se compila finalmente a JavaScript, que sigue basándose en prototipos.

Miembros estáticos

TypeScript dispone de miembros estáticos. Para acceder a ellos puede utilizarse el nombre de la clase seguido de un punto, sin crear ningún objeto.

```
class OfficeWorker {
    static memberCount: number = 0;
```

```
    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Inicialización de propiedades

Existen varias formas de inicializar las propiedades de una clase en TypeScript:

En línea:

En el siguiente ejemplo se utilizan estos valores iniciales al crear una instancia de la clase.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

En el constructor:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Mediante parámetros del constructor:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
            // There is no need to assign the values to the
properties explicitly.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Sobrecarga de métodos

La sobrecarga de métodos permite que una clase tenga varios métodos con el mismo nombre, pero con tipos o cantidades de parámetros distintos. Esto permite llamar a un método de formas diferentes según los argumentos proporcionados.

```
class MyClass {
    add(a: number, b: number): number; // Overload signature 1
    add(a: string, b: string): string; // Overload signature 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
```

```
}

const r = new MyClass();
console.log(r.add(10, 5)); // Logs 15
```

Genéricos

Los genéricos permiten crear componentes y funciones reutilizables que trabajan con varios tipos. Permiten parametrizar tipos, funciones e interfaces para que operen con tipos distintos sin especificarlos explícitamente de antemano.

Los genéricos hacen que el código sea más flexible y reutilizable.

Tipo genérico

Para definir un tipo genérico se utilizan corchetes angulares (<>) para especificar sus parámetros. Por ejemplo:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Clases genéricas

Los genéricos también pueden aplicarse a clases para que trabajen con varios tipos mediante parámetros de tipo. Esto permite crear definiciones de clase reutilizables que operan con tipos de datos distintos manteniendo la seguridad de tipos.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Restricciones genéricas

Los parámetros genéricos pueden restringirse mediante la palabra clave `extends` seguida de un tipo o interfaz que el parámetro debe satisfacer.

En el siguiente ejemplo, `T` debe tener una propiedad `length` correctamente tipada para ser válido:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Invalid
```

Una característica destacada introducida en la versión 3.4 RC es la inferencia de tipos de funciones de orden superior, que propaga los argumentos de tipo genérico:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Esta funcionalidad facilita la programación sin puntos con seguridad de tipos, habitual en la programación funcional.

Restricción contextual de genéricos

La restricción contextual de genéricos permite que el compilador restrinja el tipo de un parámetro genérico según el contexto en el que se utiliza. Resulta útil al trabajar con tipos genéricos en sentencias condicionales:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Value is narrowed down to type 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Value is narrowed down to type 'number'
        console.log(value.toFixed(2));
    }
}
```

```
process('hello'); // 5
process(3.14159); // 3.14
```

Tipos estructurales eliminados

En TypeScript, los objetos no tienen que coincidir con un tipo concreto y exacto. Si creamos un objeto que cumple los requisitos de una interfaz, podemos utilizarlo donde se exija dicha interfaz aunque no exista una conexión explícita entre ambos. Ejemplo:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Valid
```

Espacios de nombres

En TypeScript, los espacios de nombres organizan el código en contenedores lógicos, evitan colisiones de nombres y agrupan código relacionado. La palabra clave `export` permite acceder al espacio de nombres desde módulos externos.

```
export namespace MyNamespace {
    export interface MyInterface1 {
```

```
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Símbolos

Los símbolos son un tipo de datos primitivo que representa un valor inmutable cuya unicidad global se garantiza durante toda la vida del programa.

Pueden utilizarse como claves de propiedades de objetos y permiten crear propiedades no enumerables.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Ahora se permiten símbolos como claves en WeakMap y WeakSet.

Directivas de triple barra

Las directivas de triple barra son comentarios especiales que indican al compilador cómo procesar un archivo. Comienzan con tres barras consecutivas (`///`), suelen situarse al principio de un archivo TypeScript y no afectan al comportamiento en tiempo de ejecución.

Se utilizan para referenciar dependencias externas, especificar el comportamiento de carga de módulos, activar o desactivar características del compilador y otros fines. Algunos ejemplos:

Referenciar un archivo de declaración:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

Indicar el formato del módulo:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Activar opciones del compilador; en el siguiente ejemplo, el modo estricto:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Manipulación de tipos

Creación de tipos a partir de tipos

Es posible crear tipos nuevos componiendo, manipulando o transformando tipos existentes.

Tipos de intersección (`&`):

Permiten combinar varios tipos en uno solo:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection of A and B
const obj: C = { foo: 42, bar: 'hello' };
```

Tipos de unión (|):

Permiten definir un tipo que puede ser uno de varios:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Tipos mapeados:

Permiten transformar las propiedades de un tipo existente para crear otro nuevo:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Properties become read-only
```

Tipos condicionales:

Permiten crear tipos basados en determinadas condiciones:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P : never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Tipos de acceso indexado

En TypeScript es posible acceder y manipular los tipos de las propiedades de otro tipo mediante un índice, `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Tipos de utilidad

Pueden utilizarse varios tipos de utilidad integrados para manipular tipos. A continuación se muestran los más habituales:

Awaited<T>

Construye un tipo que desenvuelve recursivamente tipos Promise.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Construye un tipo con todas las propiedades de T como opcionales.

```
type Person = {
    name: string;
    age: number;
};
```

```ts
type A = Partial<Person>; // { name?: string | undefined; age?: number | undefined; }
```

Required<T>

Construye un tipo con todas las propiedades de T como obligatorias.

```ts
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Construye un tipo con todas las propiedades de T como de solo lectura.

```ts
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Invalid
```

Record<K, T>

Construye un tipo con un conjunto de propiedades K de tipo T.

```ts
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Construye un tipo seleccionando las propiedades K especificadas de T.

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Construye un tipo omitiendo las propiedades K especificadas de T.

```ts
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Construye un tipo excluyendo de T todos los valores de tipo U.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Construye un tipo extrayendo de T todos los valores de tipo U.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Construye un tipo excluyendo null y undefined de T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Extrae los tipos de los parámetros de una función de tipo T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Extrae los tipos de los parámetros de una función constructora de tipo T.

```
class Person {
    constructor(
```

```
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Extrae el tipo de retorno de una función de tipo T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Extrae el tipo de instancia de una clase de tipo T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');
```

```
person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Extrae el tipo del parámetro 'this' de una función de tipo T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

Elimina el parámetro 'this' de una función de tipo T.

```
function capitalize(this: String) {
                        return this[0].toUpperCase() + this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

Sirve como marcador de un tipo `this` contextual.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Logger> = {
    hello: function () {
```

```
        this.log('some error'); // Valid as "log" is a part of
"this".
        this.update(); // Invalid
    },
};
```

Uppercase<T>

Convierte a mayúsculas el nombre del tipo T de entrada.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Convierte a minúsculas el nombre del tipo T de entrada.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Pone en mayúscula la inicial del nombre del tipo T de entrada.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Pone en minúscula la inicial del nombre del tipo T de entrada.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer es un tipo de utilidad diseñado para bloquear la inferencia automática de tipos dentro del ámbito de una

función genérica.

Ejemplo:

```
// Automatic inference of types within the scope of a generic function.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Type here is ("a" | "b" | "c")[]
```

Con NoInfer:

```
// Example function that uses NoInfer to prevent type inference
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Error: Type Argument of type '"c"' is not assignable to parameter of type '"a" | "b"'.
```

Otros

Gestión de errores y excepciones

TypeScript permite capturar y gestionar errores mediante los mecanismos estándar de JavaScript:

Bloques try-catch-finally:

```
try {
    // Code that might throw an error
} catch (error) {
    // Handle the error
} finally {
```

```
    // Code that always executes, finally is optional
}
```

También puedes gestionar distintos tipos de errores:

```
try {
    // Code that might throw different types of errors
} catch (error) {
    if (error instanceof TypeError) {
        // Handle TypeError
    } else if (error instanceof RangeError) {
        // Handle RangeError
    } else {
        // Handle other errors
    }
}
```

Tipos de error personalizados:

Es posible especificar errores más concretos extendiendo la `class` Error:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

Clases mixin

Las clases mixin permiten combinar y componer el comportamiento de varias clases en una sola. Permiten reutilizar y ampliar funcionalidad sin necesidad de cadenas de herencia profundas.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Extend MyClass to include the behavior of Identifiable and
Selectable
interface MyClass extends Identifiable, Selectable {}

// Function to apply mixins to a class
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                  Object.defineProperty(source.prototype, name,
descriptor);
            }
```

```
        });
    });
}

// Apply the mixins to MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Características asíncronas del lenguaje

Como TypeScript es un superconjunto de JavaScript, incorpora sus características asíncronas, como:

Promesas:

Las promesas permiten gestionar operaciones asíncronas y sus resultados mediante métodos como `.then()` y `.catch()` para tratar los casos de éxito y error.

Más información: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Las palabras clave async/await ofrecen una sintaxis de aspecto más síncrono para trabajar con promesas. `async` define una función asíncrona y `await` pausa su ejecución hasta que una promesa se resuelve o rechaza.

Más información: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Las siguientes API cuentan con una buena compatibilidad en TypeScript:

API Fetch: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Iteradores y generadores

TypeScript ofrece una buena compatibilidad tanto con iteradores como con generadores.

Los iteradores son objetos que implementan el protocolo de iteración y permiten acceder uno a uno a los elementos de una colección o secuencia. Contienen un puntero al siguiente elemento y un método `next()` que devuelve el siguiente valor junto con un booleano que indica si la secuencia ha finalizado (done).

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
```

```
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Los generadores son funciones especiales definidas mediante la sintaxis `function*` que simplifican la creación de iteradores. Utilizan `yield` para definir la secuencia de valores y pausan y reanudan automáticamente la ejecución cuando se solicitan valores.

Los generadores facilitan la creación de iteradores y resultan especialmente útiles con secuencias grandes o infinitas.

Ejemplo:

```
function* numberGenerator(start: number, end: number):
Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript también admite iteradores y generadores asíncronos.

Más información:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

Referencia de TsDocs y JSDoc

Al trabajar con una base de código JavaScript, los comentarios JSDoc con anotaciones adicionales pueden ayudar a TypeScript a inferir el tipo correcto.

Ejemplo:

```
/**
 * Computes the power of a given number
 * @constructor
 * @param {number} base - The base value of the expression
```

```
 * @param {number} exponent - The exponent value of the
expression
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent:
number): number
```

La documentación completa está disponible en: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Desde la versión 3.7 es posible generar definiciones de tipos .d.ts a partir de la sintaxis JSDoc de JavaScript. Encontrarás más información en: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Los paquetes de la organización @types siguen una convención especial de nombres para proporcionar definiciones de tipos de bibliotecas o módulos JavaScript existentes. Por ejemplo:

```
npm install --save-dev @types/lodash
```

instalará las definiciones de tipos de `lodash` en el proyecto actual.

Para contribuir a las definiciones de un paquete `@types`, envía una solicitud de incorporación de cambios a https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) es una extensión de la sintaxis de JavaScript que permite escribir código similar a HTML dentro de archivos JavaScript o TypeScript. Se utiliza habitualmente en React para definir la estructura HTML.

TypeScript amplía las capacidades de JSX mediante comprobación de tipos y análisis estático.

Para utilizar JSX debes establecer la opción `jsx` del compilador en `tsconfig.json`. Dos opciones habituales son:

- "preserve": emite archivos .jsx con JSX sin cambios. Indica a TypeScript que no transforme la sintaxis JSX durante la compilación. Puede utilizarse si otra herramienta, como Babel, realiza la transformación.
- "react": activa la transformación JSX integrada de TypeScript. Se utilizará React.createElement.

Todas las opciones están disponibles en: https://www.typescriptlang.org/tsconfig#jsx

Módulos ES6

TypeScript admite ES6 (ECMAScript 2015) y muchas versiones posteriores. Esto permite utilizar sintaxis de ES6, como funciones flecha, literales de plantilla, clases, módulos y desestructuración, entre otras características.

Para activar las características de ES6 en el proyecto, puedes especificar la propiedad `target` en tsconfig.json.

Ejemplo de configuración:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Operador de exponenciación de ES7

El operador de exponenciación (`**`) calcula el valor obtenido al elevar el primer operando a la potencia del segundo. Funciona de forma similar a `Math.pow()`, pero también acepta valores `bigint` como operandos. TypeScript admite completamente este operador al establecer `target` en `es2016` o una versión superior en tsconfig.json.

```
console.log(2 ** (2 ** 2)); // 16
```

La sentencia for-await-of

Esta característica de JavaScript, totalmente compatible con TypeScript, permite recorrer objetos iterables asíncronos cuando la versión de destino es `es2018`.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
```

```
        console.log(num);
    }
})();
```

Metapropiedad new target

La metapropiedad `new.target` permite determinar si una función o un constructor se invocó mediante el operador new. Así se puede detectar si un objeto se creó como resultado de una llamada a un constructor.

```
class Parent {
    constructor() {
            console.log(new.target);  // Logs the constructor function used to create an instance
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Expresiones de importación dinámica

Es posible cargar módulos de forma condicional o diferida bajo demanda mediante la propuesta de importación dinámica de ECMAScript, compatible con TypeScript.

La sintaxis de las expresiones de importación dinámica es la siguiente:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
          const widget = await import('./widget'); // Dynamic
import
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

Este comando inicia el compilador de TypeScript con el parámetro `--watch`, que recompila automáticamente los archivos TypeScript cuando se modifican.

```
tsc --watch
```

Desde TypeScript 4.9, la supervisión de archivos utiliza principalmente eventos del sistema de archivos y recurre automáticamente al sondeo si no puede establecerse un observador basado en eventos.

Operador de aserción no nula

El operador de aserción no nula (! posfijo), también llamado aserción de asignación definida, permite afirmar que una variable o propiedad no es null ni undefined aunque el análisis estático sugiera lo contrario. Así pueden eliminarse comprobaciones explícitas.

```
type Person = {
    name: string;
};
```

```
const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Declaraciones con valores predeterminados

Las declaraciones con valores predeterminados se utilizan cuando se asigna un valor predeterminado a una variable o parámetro. Si no se proporciona ningún valor, se utilizará el predeterminado.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Encadenamiento opcional

El operador de encadenamiento opcional `?.` funciona como el operador de punto (`.`) al acceder a propiedades o métodos. Sin embargo, gestiona valores null o undefined terminando la expresión y devolviendo `undefined` en lugar de generar un error.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};
```

```
console.log(person.address?.city); // undefined
```

Operador de fusión de valores nulos

El operador de fusión de valores nulos `??` devuelve el valor del lado derecho si el izquierdo es `null` o `undefined`; en caso contrario, devuelve el del lado izquierdo.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Tipos literales de plantilla

Los tipos literales de plantilla permiten manipular valores de cadena en el nivel de tipos y generar tipos de cadena nuevos a partir de otros existentes. Resultan útiles para crear tipos más expresivos y precisos mediante operaciones con cadenas.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Sobrecarga de funciones

La sobrecarga de funciones permite definir varias firmas para un mismo nombre de función, cada una con distintos tipos de parámetros y de retorno. Al llamar a una función sobrecargada,

TypeScript utiliza los argumentos proporcionados para determinar la firma correcta:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Tipos recursivos

Un tipo recursivo puede hacer referencia a sí mismo. Resulta útil para definir estructuras de datos jerárquicas o recursivas, con anidamiento potencialmente infinito, como listas enlazadas, árboles y grafos.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Tipos condicionales recursivos

En TypeScript es posible definir relaciones de tipos complejas mediante lógica y recursión. Veámoslo en términos sencillos:

Los tipos condicionales permiten definir tipos basados en condiciones booleanas:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

La recursión consiste en una definición de tipo que hace referencia a sí misma:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

Los tipos condicionales recursivos combinan la lógica condicional y la recursión. Una definición puede depender de sí misma mediante lógica condicional y crear relaciones de tipos complejas y flexibles.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 | 6
```

Compatibilidad con módulos ECMAScript en Node

Node.js añadió compatibilidad con módulos ECMAScript en la versión 15.3.0 y TypeScript la ofrece para Node.js desde la

versión 4.7. Puede activarse mediante la propiedad `module` con el valor `nodenext` en tsconfig.json. Por ejemplo:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js admite dos extensiones para módulos: `.mjs` para módulos ES y `.cjs` para CommonJS. Las extensiones equivalentes de TypeScript son `.mts` y `.cts`. Al transpilarlos a JavaScript, el compilador crea archivos `.mjs` y `.cjs`.

Para utilizar módulos ES puedes establecer la propiedad `type` en "module" en package.json. Esto indica a Node.js que trate el proyecto como un proyecto de módulos ES.

TypeScript también admite declaraciones de tipos en archivos `.d.ts`. Estos archivos proporcionan información de tipos para bibliotecas o módulos escritos en TypeScript y permiten que otros desarrolladores los utilicen con las funciones de comprobación de tipos y finalización automática de TypeScript.

Funciones de aserción

En TypeScript, las funciones de aserción permiten verificar una condición concreta a partir de su valor de retorno. En su forma más sencilla, examinan un predicado y generan un error cuando se evalúa como falso.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
```

```
        throw new Error('Not a number');
    }
}
```

También puede declararse como expresión de función:

```
type AssertIsNumber = (value: unknown) => asserts value is number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Las funciones de aserción se parecen a las guardas de tipo. Estas se introdujeron para realizar comprobaciones en tiempo de ejecución y garantizar el tipo de un valor dentro de un ámbito concreto. En particular, una guarda de tipo evalúa un predicado y devuelve un booleano que indica si es verdadero o falso. Las funciones de aserción, en cambio, pretenden lanzar un error en lugar de devolver false cuando no se satisface el predicado.

Ejemplo de guarda de tipo:

```
const isNumber = (value: unknown): value is number => typeof value === 'number';
```

Tipos de tupla variádicos

Los tipos de tupla variádicos se introdujeron en TypeScript 4.0. Empecemos recordando qué es una tupla:

Un tipo de tupla es un array de longitud definida en el que se conoce el tipo de cada elemento:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

El término «variádico» significa aridad indefinida (aceptar un número variable de argumentos).

Una tupla variádica conserva todas las propiedades anteriores, pero su forma exacta aún no está definida:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

En el código anterior, la forma de la tupla viene definida por el genérico `T` proporcionado.

Las tuplas variádicas pueden aceptar varios genéricos, lo que las hace muy flexibles:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

Con las nuevas tuplas variádicas podemos utilizar:

- Las expansiones de la sintaxis de tipos de tupla ahora pueden ser genéricas, por lo que podemos representar operaciones de orden superior sobre tuplas y arrays aunque no conozcamos los tipos reales.
- Los elementos rest pueden aparecer en cualquier posición de una tupla.

Ejemplo:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Tipos envoltorio

Los tipos envoltorio son objetos contenedores que representan tipos primitivos como objetos. Proporcionan funcionalidad y métodos adicionales que no están disponibles directamente en los valores primitivos.

Al acceder a un método como `charAt` o `normalize` sobre un primitivo `string`, JavaScript lo envuelve en un objeto `String`, llama al método y después descarta el objeto.

Demostración:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript representa esta diferencia proporcionando tipos distintos para los primitivos y sus objetos contenedores:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Los tipos envoltorio no suelen ser necesarios. Evítalos y utiliza los tipos primitivos; por ejemplo, `string` en lugar de `String`.

Covarianza y contravarianza en TypeScript

La covarianza y la contravarianza describen las relaciones entre tipos dentro de los tipos genéricos. Un `Dog` es un `Animal`, pero no todos los `Animal` son `Dog`.

Covarianza (producción): una función que devuelve un `Dog` puede sustituir a una que devuelve un `Animal`. Todos los perros son animales, pero una función que devuelve cualquier animal podría devolver un gato.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Valid
produceDog = produceAnimal; // Error
```

Contravarianza (consumo): una función que acepta cualquier `Animal` puede sustituir a una que acepta un `Dog`. También admite perros; una función que solo acepta perros no puede procesar todos los animales.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Valid
consumeAnimal = consumeDog; // Error with strictFunctionTypes
```

Las flechas indican el sentido de las asignaciones seguras:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Arrays mutables: TypeScript también permite asignar `Dog[]` a `Animal[]`. Ambas variables se refieren al mismo array; insertar un `Animal` mediante el tipo más general puede añadir un elemento que no es un perro.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Allowed, but unsafe
// dogs now contains an Animal that is not a Dog.
```

Opción del compilador: con `strictFunctionTypes`, los parámetros de funciones independientes se comprueban contravariantemente. Sin esta opción, TypeScript puede aceptar ambos sentidos (bivarianza), aunque no sean seguros. Los

parámetros declarados en métodos y constructores son una excepción.

Anotaciones opcionales de varianza para parámetros de tipo

Desde TypeScript 4.7, `out` indica covarianza, `in` contravarianza e `in out` invariancia. TypeScript suele inferir la varianza. Las anotaciones deben corresponder al uso del parámetro genérico, no cambiarlo arbitrariamente.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Firmas de índice con patrones de cadenas de plantilla

Las firmas de índice con patrones de cadenas de plantilla permiten definir firmas flexibles mediante patrones de cadenas. Esta característica permite crear objetos indexables con patrones concretos de claves de cadena y proporciona más control y precisión al acceder y manipular propiedades.

Desde la versión 4.4, TypeScript admite firmas de índice para símbolos y patrones de cadenas de plantilla.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};
```

```
const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

El operador satisfies

El operador `satisfies` permite comprobar si un tipo satisface una interfaz o condición concreta. Garantiza que tenga todas las propiedades y métodos obligatorios de una interfaz determinada y que una variable se ajuste a la definición de un tipo. Este es un ejemplo:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Type Annotation using `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// In the following lines, TypeScript won't be able to infer properly
user.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user.nickName; // string | string[] | undefined
```

```
// Type assertion using `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Here too, TypeScript won't be able to infer properly
user2.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user2.nickName; // string | string[] | undefined

// Using the `satisfies` operator we can properly infer the types now
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log);   //   TypeScript   infers correctly: string[]
user3.nickName; // TypeScript infers correctly: undefined
```

Importaciones y exportaciones solo de tipos

Las importaciones y exportaciones solo de tipos permiten importar o exportar tipos sin los valores o funciones asociados. Esto puede ayudar a reducir el tamaño del paquete.

Para utilizarlas, puedes emplear la palabra clave `import type`.

TypeScript permite utilizar extensiones de archivos de declaración e implementación (.ts, .mts, .cts y .tsx) en importaciones solo de tipos, independientemente de la configuración de `allowImportingTsExtensions`.

Por ejemplo:

```
import type { House } from './house.ts';
```

Se admiten las siguientes formas:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

Declaración using y gestión explícita de recursos

Una declaración `using` es una vinculación inmutable con ámbito de bloque, similar a `const`, que gestiona recursos desechables. Al inicializarse con un valor, se registra su método `Symbol.dispose`, que se ejecuta al salir del bloque contenedor.

Se basa en la gestión de recursos de ECMAScript, útil para realizar tareas de limpieza esenciales después de crear objetos, como cerrar conexiones, eliminar archivos y liberar memoria.

Notas:

- Debido a su reciente introducción en TypeScript 5.2, la mayoría de los entornos no ofrece compatibilidad nativa. Necesitarás polyfills para `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack` y `SuppressedError`.
- Además, deberás configurar tsconfig.json de la siguiente forma:

```
{
    "compilerOptions": {
```

```
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Ejemplo:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Simple polyfill

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Resource is declared
    console.log(2);
} // Resource is disposed (e.g., `work[Symbol.dispose]()` is
evaluated)

console.log(3);
```

El código muestra:

```
1
2
disposed
3
```

Un recurso desechable debe ajustarse a la interfaz `Disposable`:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

Las declaraciones `using` registran las operaciones de liberación en una pila para garantizar que los recursos se liberen en orden inverso al de su declaración:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // disposes `C`, then `B`, then `A`.
```

Se garantiza que los recursos se liberen aunque el código posterior o las excepciones interrumpan la ejecución. La liberación puede lanzar una excepción y ocultar otra. Para conservar información sobre los errores suprimidos se introduce la excepción nativa `SuppressedError`.

Declaración await using

Una declaración `await using` gestiona un recurso desechable de forma asíncrona. El valor debe tener un método `Symbol.asyncDispose`, que se esperará al final del bloque.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Resource is declared
}     //     Resource     is     disposed     (e.g.,     `await
work[Symbol.asyncDispose]()` is evaluated)
```

Un recurso desechable de forma asíncrona debe ajustarse a la interfaz `Disposable` o `AsyncDisposable`:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // Simple polyfill

class DatabaseConnection implements AsyncDisposable {
     // A method that is called when the object is disposed asynchronously
    [Symbol.asyncDispose]() {
        // Close the connection and return a promise
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
            await new Promise(resolve => setTimeout(resolve, 1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
     // Create a new connection and dispose it asynchronously when it goes out of scope
      await using connection = new DatabaseConnection(); // Resource is declared
    console.log('Doing some work...');
}     //     Resource     is     disposed     (e.g.,     `await connection[Symbol.asyncDispose]()` is evaluated)

doWork();
```

El código muestra:

```
Doing some work...
Closing the connection...
Connection closed.
```

Las declaraciones `using` y `await using` se permiten en las sentencias `for`, `for-in`, `for-of`, `for-await-of` y `switch`.

Atributos de importación

Los atributos de importación de TypeScript 5.3 indican al entorno cómo gestionar módulos como JSON. Mejoran la seguridad al hacer explícitas las importaciones y se ajustan a la Política de Seguridad de Contenidos (CSP) para cargar recursos de forma más segura. TypeScript comprueba su validez, pero deja su interpretación al entorno de ejecución.

Ejemplo:

```
import config from './config.json' with { type: 'json' };
```

Con importación dinámica:

```
const config = import('./config.json', { with: { type: 'json' }
});
```

Comprobación de la sintaxis de expresiones regulares

Desde TypeScript 5.5.4 se comprueban durante la compilación errores habituales en literales de expresiones regulares, como sintaxis no válida, referencias inversas incorrectas o características incompatibles con la versión de JavaScript de destino. Esto permite detectar errores antes, pero no comprueba las cadenas pasadas a new RegExp(“…”).

```
let r = /(a)\2/; // Error: This backreference refers to a group
that does not exist.
```

import defer

`import defer` permite cargar un módulo y retrasar su ejecución hasta que se utiliza alguna de sus exportaciones. Esto evita trabajo y efectos secundarios innecesarios.

- Solo funciona con `import defer * as name from "module"`.
- El código solo se ejecuta al acceder a una exportación.